UNIVERSITÀ POLITECNICA DELLE MARCHE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

Insegnamento di New Generation Databases



Presentato da:

Andrea Altieri

Andrea Flaiani

Docente:

Prof.ssa Claudia Diamantini

ANOMALY DETECTION SU DATI BIOMETRICI

RELAZIONE DI PROGETTO

Anno Accademico 2026/2027

Indice

1 Introduzione e dominio applicativo **1**

1.1 Contesto e motivazioni del progetto . . . 1
1.2 Il dominio clinico e il dataset VitalDB . . . 1
1.3 Obiettivi e struttura dell'elaborato . . . 3

2 Architettura della data pipeline **4**

2.1 Il paradigma della medallion architecture . . . 4
2.2 Layer Bronze per l'ingestione e la storicizzazione dei dati grezzi . . . 5
2.3 Layer Silver per la bonifica e il partizionamento dei dati . . . 6
2.4 Evoluzione del modello dati da tabella piatta a documento BSON . . . 9

3 Archiviazione e governance con MongoDB Time Series **11**

3.1 Layer Gold e archiviazione con MongoDB Time Series . . . 11
3.2 Benchmark prestazionale e di occupazione dello storage . . . 12
3.3 Data governance artigianale in ottica data lakehouse . . . 14
3.3.1 Catalogo, macchina a stati e provenienza (collezione registry) . . . 14
3.3.2 Metadati descrittivi delle tracce e sincronizzazione automatica (track_metadata) . . . 16
3.3.3 Limiti delle Time Series, protocollo a due fasi e validazione applicativa . . . 16
3.4 Pipeline di aggregazione analitica e ottimizzazione delle query . . . 17
3.4.1 Downsampling temporale efficiente lato database con $dateTrunc . . . 17
3.4.2 Ottimizzazione dell'accesso anagrafico ed eliminazione del problema N+1 . . . 18
3.5 Uso dei metadati per la data quality e analisi critica . . . 19

4 Modelli di anomaly detection **21**

4.1 Approccio ibrido e multilivello al rilevamento di anomalie . . . 21
4.2 Regole cliniche dominio-specifiche e filtri di freschezza . . . 21
4.2.1 Shock Index clinico . . . 22
4.2.2 Ipotensione severa concomitante a ipossiemia . . . 22
4.3 Ingegnerizzazione delle feature temporali . . . 22
4.4 Modelli di machine learning non supervisionato . . . 23
4.4.1 Isolation Forest e semantica della soglia di contaminazione . . . 24
4.4.2 LSTM Autoencoder neurale (PyTorch) e scalabilità zero-copy . . . 24
4.5 Valutazione empirica, metriche e concordanza . . . 27
4.6 Persistenza sicura delle anomalie su MongoDB (Insert-Before-Delete) . . . 29

5 Ingegnerizzazione del sistema, interfaccia grafica e conclusioni **31**

5.1 Backend REST API con FastAPI . . . 31
5.2 Dashboard web e interfaccia utente interattiva . . . 33
5.2.1 Vista di telemetria clinica e anomaly detection . . . 33

5.2.2 Vista di data lakehouse, governance e benchmark 35
5.3 Containerizzazione Docker, test automatizzati e manutenzione 35
5.3.1 Verifica sperimentale automatizzata (Pytest) e strumenti di reset 36
5.4 Conclusioni, limiti e sviluppi futuri 37

CAPITOLO 1

Introduzione e dominio applicativo

1.1 Contesto e motivazioni del progetto

Il monitoraggio fisiologico continuo in ambito operatorio e nei reparti di terapia intensiva genera una mole imponente di dati temporali ad alta frequenza. Durante un intervento chirurgico, monitor multiparametrici registrano senza sosta segnali biometrici vitali: battito cardiaco, concentrazione di ossigeno nel sangue e andamento della pressione arteriosa. La disponibilità di queste serie storiche dischiude grandi opportunità per la medicina computazionale, ma al contempo pone sfide ingegneristiche complesse legate alla gestione, alla qualità e alla tempestività dell'analisi del dato.

I dati clinici grezzi raccolti in sala operatoria sono intrinsecamente rumorosi e disomogenei. Il movimento del paziente o dell'operatore, il distacco accidentale degli elettrodi e le interferenze strumentali provocano artefatti evidenti, dispersioni e lunghi intervalli di valori mancanti. A ciò si aggiunge una marcata asimmetria nelle frequenze di campionamento: mentre la frequenza cardiaca e la saturazione periferica dell'ossigeno vengono campionate con cadenza quasi continua ogni uno o due secondi, le misurazioni pressorie non invasive avvengono a intervalli discreti e dilatati, tipicamente ogni due o cinque minuti, a causa dei tempi necessari per il gonfiaggio e lo sgonfiaggio del bracciale pneumatico.

In un simile scenario, le tradizionali architetture di archiviazione faticano a garantire al tempo stesso l'integrità del dato, l'efficienza dello storage e la reattività richiesta da interrogazioni analitiche complesse. Il presente progetto si colloca nell'alveo dei database di nuova generazione, affrontando l'ingegnerizzazione completa di una piattaforma per l'ingestione, la bonifica, la catalogazione e l'analisi automatizzata di serie storiche biometriche. L'obiettivo primario consiste nel superare la frammentazione tipica dei sistemi ospedalieri mediante un flusso strutturato a stadi, fondato sui principi del **data lakehouse**, associando una rigorosa disciplina di **data governance** all'impiego congiunto di regole cliniche ed algoritmi di **machine learning** per l'individuazione precoce delle instabilità emodinamiche.

1.2 Il dominio clinico e il dataset VitalDB

Per lo sviluppo e la validazione del sistema è stato selezionato il dataset clinico aperto **VitalDB**, un archivio ad accesso pubblico contenente registrazioni fisiologiche ad alta fedeltà relative a oltre 6.300 interventi chirurgici non cardiaci e cardiaci, raccolte presso il Seoul National University Hospital tra il 2016 e il 2019.

La scelta di VitalDB è maturata a valle di un confronto comparativo con il dataset alternativo MIMIC-IV. Sebbene quest'ultimo rappresenti un punto di riferimento altrettanto autorevole nella letteratura medica, esso impone un percorso articolato di abilitazione attraverso la piattaforma PhysioNet, con corsi accreditati di etica e protezione dei dati che avrebbero dilatato i tempi iniziali del progetto. Al contrario, VitalDB fornisce un accesso immediato e programmatico mediante una libreria ufficiale in linguaggio Python, garantendo al contempo una granularità temporale e una varietà di parametri pienamente equivalenti e ideali per la sperimentazione di architetture dati avanzate.

In pieno accordo con gli obiettivi del corso, l'attenzione è stata deliberatamente circoscritta all'elaborazione dei parametri biometrici numerici già estratti e standardizzati dalla strumentazione di sala, escludendo le forme d'onda grezze ad altissima frequenza come i tracciati elettrocardiografici o fotopletismografici campionati a centinaia di hertz. L'elaborazione di segnali grezzi avrebbe infatti richiesto una complessa componente di **digital signal processing** e stima biomedica non centrale rispetto alle finalità dell'elaborato, che vertono primariamente sul **data engineering**, sulla modellazione dei database a documenti e sulle tecniche di **anomaly detection**.

L'indagine sperimentale si concentra su cinque metriche vitali primarie registrate dal modulo di monitoraggio `Solar8000`: il parametro `Solar8000/HR`, che quantifica la frequenza cardiaca espressa in battiti per minuto (bpm); il parametro `Solar8000/PLETH_SPO2`, che rileva la saturazione d'ossigeno dell'emoglobina periferica espressa in percentuale; il parametro `Solar8000/NIBP_SBP`, che misura la pressione arteriosa sistolica non invasiva espressa in millimetri di mercurio (mmHg); il parametro `Solar8000/NIBP_DBP`, che registra la corrispondente pressione arteriosa diastolica in millimetri di mercurio; il parametro `Solar8000/NIBP_MBP`, che esprime la pressione arteriosa media, grandezza fondamentale per stimare la perfusione d'organo.

Dal punto di vista fisiopatologico, la pressione arteriosa media non rappresenta una semplice media aritmetica, bensì una stima integrata nel ciclo cardiaco, approssimata dalla relazione clinica standard:

$$MBP \approx DBP + \frac{1}{3}(SBP - DBP)$$

Nel paziente chirurgico sottoposto ad anestesia generale, la vasodilatazione farmacologica e l'azione deprimente sul miocardio inducono frequentemente cali pressori improvvisi. Il mantenimento di un valore di $MBP \geq 65$ mmHg costituisce il target emodinamico raccomandato dalle linee guida internazionali per scongiurare danni ischemici acuti al circolo renale, coronarico e cerebrale.

L'eterogeneità temporale tra le sorgenti sensoristiche costituisce la sfida ingegneristica centrale del progetto: mentre la frequenza cardiaca e la saturazione periferica sono acquisite con cadenza continua a 1 Hz tramite elettrodi ed emettitori ottici fotopletismografici, le misurazioni pressorie non invasive impiegano un bracciale pneumatico oscillometrico che necessita di tempi tecnici di gonfiaggio e sgonfiaggio compresi tra 20 e 40 secondi, ripetuti a intervalli programmati ogni due o cinque minuti. Ne deriva una matrice di serie storiche caratterizzata da forte asincronia e valori mancanti prolungati.

Queste rilevazioni fisiologiche sono affiancate da metadati clinici e referti di laboratorio perioperatori scaricati contestualmente, tra cui l'età, il sesso, i parametri ematochimici e il reparto chirurgico di afferenza del paziente (`department`, ad esempio chirurgia generale, toracica o ginecologica). Il campione sperimentale selezionato per la validazione su larga scala comprende 500 interventi chirurgici completi per un totale di 5.402.386 secondi di tracciato attivo nel livello **Gold** (corrispondenti a oltre 1.500 ore complessive di sala operatoria, estratte a partire da 5.565.384 secondi grezzi registrati nel livello **Bronze**). L'integrazione di questi attributi contestuali risulta determinante non solo per consentire aggregazioni epidemiologiche, ma anche per guidare il partizionamento fisico dei dati e ottimizzare le strategie di indicizzazione all'interno della base di dati.

Sintesi della coorte e delle sorgenti sensoristiche

La base dati sperimentale raccoglie 500 interventi chirurgici per un totale di 5.402.386 secondi bonificati (> 1.500 ore di monitoraggio continuo). La telemetria integra segnali continui a 1 Hz da fotopletismografia ottica (frequenza cardiaca e saturazione d'ossigeno)

con rilievi emodinamici intermittenti a bracciale oscillometrico (pressioni sistolica, diastolica e media), ancorando le soglie di allarme al limite critico di ipoperfusione d'organo ($MBP < 65$ mmHg) e verificando la freschezza temporale di ciascuna lettura pressoria.

1.3 Obiettivi e struttura dell'elaborato

Il lavoro svolto si articola attorno a cinque direttrici cardine che guidano l'intero ciclo di vita del dato clinico: la costruzione di una pipeline di raffinamento progressivo strutturata secondo il paradigma della **Medallion Architecture**, articolata nei livelli **Bronze**, **Silver** e **Gold** e coordinata da un identificativo univoco di esecuzione (`run_id`) e di versione (`pipeline_version`); l'archiviazione scalabile delle serie temporali mediante le **Time Series Collection** native di MongoDB 8.0, sfruttando il raggruppamento automatico a **bucket** compressi per massimizzare la densità dello storage e accelerare le query temporali; l'implementazione di un modello di **data governance** artigianale ispirato ai **data lakehouse**, comprendente una fonte unica di verità per i limiti fisiologici (`TRACK_SPECS`), un catalogo di registrazione delle serie con protocollo di pubblicazione a due fasi (`loading` $\rightarrow$ `committed`), tracciamento della provenienza (**data lineage**) cross-layer e l'impiego strutturato dei metadati per monitorare la qualità del dato; lo sviluppo di un'architettura multilivello di **anomaly detection** consapevole della qualità e della freschezza del dato, capace di coniugare euristiche cliniche dominio-specifiche (lo **Shock Index** e la condizione di ipotensione severa) con modelli di **machine learning** non supervisionato (l'algoritmo ad alberi **Isolation Forest** e un **autoencoder** neurale ricorrente basato su PyTorch); l'ingegnerizzazione completa dell'ecosistema attraverso un backend REST realizzato con il framework **FastAPI**, una dashboard interattiva ricca di componenti visivi per la telemetria clinica, una suite automatizzata di 38 test unitari e l'orchestrazione trasparente dei servizi mediante container Docker.

La trattazione è organizzata come segue: nel Capitolo 2 viene approfondita l'architettura della pipeline e il processo di pulizia, tracciamento delle imputazioni e partizionamento nei livelli **Bronze** e **Silver**; nel Capitolo 3 vengono presentate l'archiviazione nel layer **Gold** su MongoDB, i benchmark di memorizzazione e l'intera impalcatura di **data governance**; nel Capitolo 4 sono esposti i modelli matematici e clinici di rilevamento delle anomalie con la relativa valutazione quantitativa; infine, nel Capitolo 5 sono descritti la progettazione del backend, l'interfaccia utente web, la verifica sperimentale tramite test unitari, la containerizzazione con Docker e le prospettive di sviluppo futuro.

CAPITOLO 2

Architettura della data pipeline

2.1 Il paradigma della medallion architecture

Nel trattamento di serie temporali complesse, l'adozione di un'architettura dati strutturata risulta indispensabile per assicurare la tracciabilità delle trasformazioni, l'integrità dei valori e la riproducibilità degli esperimenti. Nel presente progetto è stata implementata la **Medallion Architecture**, un pattern architetturale ampiamente diffuso nell'ingegneria dei **data lakehouse** che organizza il flusso di elaborazione in tre stadi logici a qualità progressivamente crescente, denominati convenzionalmente **Bronze**, **Silver** e **Gold**.

Come illustrato nella Figura 2.1, ciascun livello assolve a un compito ben definito all'interno della catena di raffinamento del dato: il livello **Bronze** accoglie i dati grezzi estratti direttamente dalla sorgente clinica, salvaguardandoli nella loro forma originaria senza alcuna alterazione; il livello **Silver** applica algoritmi di bonifica, normalizzazione, filtraggio degli artefatti sensoristici e partizionamento tematico per reparto; il livello **Gold** struttura i dati in documenti conformi al modello a serie storiche di MongoDB, predisponendoli al consumo efficiente da parte delle interfacce grafiche, dei servizi REST e dei modelli di **machine learning**.



Figura 2.1: Panoramica dell'architettura Medallion e dei livelli Bronze, Silver e Gold generata in TikZ.

L'adozione di questa architettura multi-stadio garantisce che gli errori introdotti in un passaggio intermedio possano essere isolati e corretti rieseguendo le trasformazioni a partire dai livelli precedenti, preservando al contempo una netta separazione delle responsabilità tra ingestione, trasformazione analitica e fruizione finale del dato.

L'intera catena di elaborazione, schematizzata nella Figura 2.2, si articola in cinque fasi operative

sequenziali che connettono l'estrazione remota alla visualizzazione telemetrica.

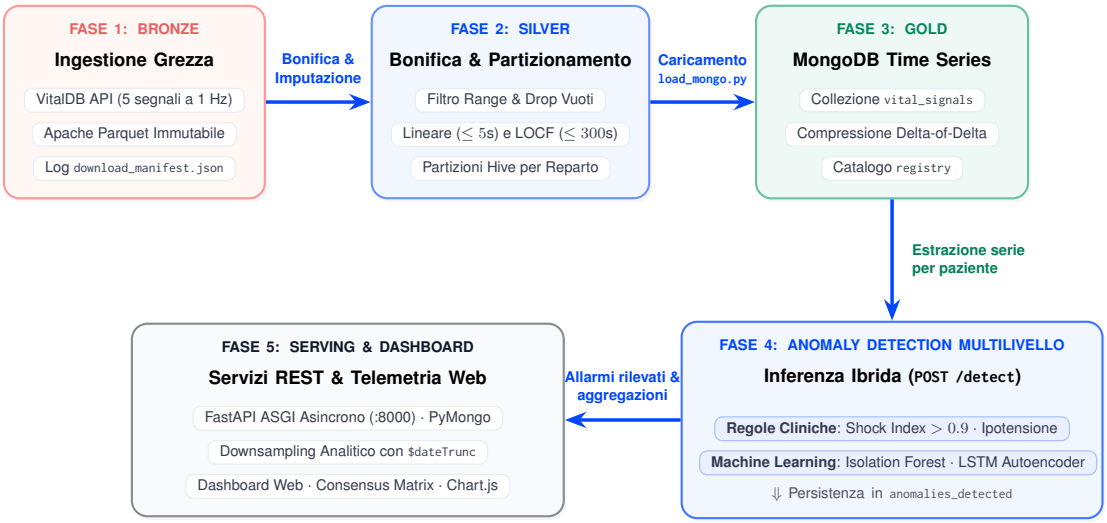


Figura 2.2: Architettura della pipeline di esecuzione end-to-end dall'ingestione alla visualizzazione.

2.2 Layer Bronze per l'ingestione e la storicizzazione dei dati grezzi

Il livello **Bronze** costituisce il punto di ingresso dell'intera infrastruttura. Il modulo di acquisizione, implementato nello script `src/bronze/download.py`, interagisce direttamente con l'archivio centrale di VitalDB sfruttando l'API Python ufficiale.

L'ingestione seleziona in modo automatico i casi clinici tramite la funzione `vitaldb.find_cases`, che individua i pazienti per i quali risultano registrate contemporaneamente le cinque metriche fisiologiche di interesse: la frequenza cardiaca, la saturazione d'ossigeno e le tre componenti pressorie. Per ciascun caso individuato, la procedura scarica i tracciati temporali mediante `vitaldb.load_case` impostando una frequenza nominale di campionamento pari a un secondo (`interval=1.0`).

I tracciati acquisiti vengono persistiti in locale nella cartella `data/bronze/` all'interno di file conformi allo standard Apache Parquet, con convenzione di denominazione `case_<ID>.parquet`. La scelta del formato Parquet per lo strato grezzo rispetto al testo semplice CSV è motivata dalla natura colonnare e compressa del formato, che riduce in misura rilevante lo spazio occupato su disco e memorizza in modo nativo i tipi di dato numerici a virgola mobile a 64 bit, scongiurando qualsiasi perdita di precisione numerica o ambiguità di codifica.

Contestualmente all'acquisizione dei segnali temporali, la procedura estrae le informazioni cliniche e perioperatorie salvandole nel file ausiliario `data/bronze/clinical_data.parquet`, che raccoglie attributi fondamentali quali l'età del paziente, il sesso, l'indice di massa corporea e il reparto chirurgico di ricovero. Poiché l'implementazione predefinita della funzione `vitaldb.load_clinical_data()` nella libreria Python ufficiale presenta un noto difetto che restituisce una tabella vuota se invocata senza argomenti espliciti, il modulo `download.py` interroga in via prioritaria l'endpoint REST ufficiale (`https://api.vitaldb.net/cases`), mantenendo l'SDK parametrizzato come meccanismo di fallback. Qualora entrambe le modalità dovessero fallire o restituire zero record, la pipeline applica una politica **fail-fast** interrompendo immediatamente l'esecuzione con codice di uscita non nullo (`sys.exit(1)`): questo blocco preventivo è motivato dal fatto che un'ingestione priva della mappatura dei reparti chirurgici degraderebbe silenziosamente l'intero partizionamento del successivo livello **Silver**, riversando tutti i pazienti nella partizione residuale `department=UNKNOWN`. A completamento del contesto clinico, la funzione `download_lab_data` scarica

e filtra sui soli `case_id` della coorte anche i referti ematochimici perioperatori, persistendoli in `data/bronze/lab_data.parquet`.

In conformità con i principi di **data governance**, il livello **Bronze** adotta una politica di immutabilità rigorosa: i file vengono memorizzati esattamente nello stato in cui sono stati emessi dalla sorgente remota, senza alcuna rimozione di valori nulli o alterazione temporale. Per garantire una tracciabilità formale (**data lineage**) lungo l'intera catena, l'avvio dello script `download.py` genera un identificativo univoco di esecuzione (`run_id`, persistito in `data/.current_run_id` tramite `config.get_or_create_run_id`) che viene inciso, insieme alla versione della pipeline (`pipeline_version = "1.1.0"`), all'interno del registro strutturato `download_manifest.json`. Tale manifest censisce per ogni singolo caso l'esito puntuale (`downloaded`, `skipped_existing` o `failed`), il numero di campioni e il peso in byte. Inoltre, prima di saltare il download di un file Parquet già presente su disco (`skipped_existing`), la funzione `download_case` ispeziona la struttura del file per verificare che contenga effettivamente tutte le cinque tracce richieste più la colonna `Time` e che il passo di campionamento coincida con `SAMPLING_INTERVAL` (1.0 s): questa verifica impedisce che file residui generati da esecuzioni precedenti con configurazioni o frequenze differenti inquinino silenziosamente la corsa corrente.

2.3 Layer Silver per la bonifica e il partizionamento dei dati

I dati clinici immagazzinati nel livello **Bronze** non possono essere impiegati direttamente per l'addestramento di modelli predittivi o per il monitoraggio in tempo reale a causa della notevole quantità di rumore, discontinuità e valori nulli. Il livello **Silver**, orchestrato dallo script `src/silver/clean.py`, si fa carico della bonifica del dato, della gestione dei valori mancanti, dell'etichettatura degli outlier fisiologici, del tracciamento esplicito delle imputazioni per singolo canale e della riorganizzazione fisica dei file su file system. Prima di avviare l'elaborazione, la funzione `process_all_cases` riallinea automaticamente il perimetro dei casi rispetto al `download_manifest.json` attivo (o al limite configurato `MAX_CASES = 500`), rimuovendo eventuali file Parquet **Silver** orfani di esecuzioni precedenti con perimetro differente, così da garantire che tutti i layer descrivano esattamente la medesima coorte.

Un requisito metodologico fondamentale nella pulizia di serie temporali a 1 Hz consiste nel preservare l'integrità della griglia temporale durante le operazioni di interpolazione. Se si eliminassero in apertura le righe completamente vuote (`dropna(how='all')`), due campioni originariamente separati da un'interruzione di molti minuti diverrebbero artificialmente adiacenti nell'indice del DataFrame, falsando il significato fisico delle finestre temporali espresse in numero di campioni. Per questa ragione, nel modulo `clean.py` la griglia temporale a 1 Hz viene mantenuta intatta durante tutte le fasi di controllo e imputazione, posticipando lo scarto delle righe prive di qualsiasi misurazione su tutte e cinque le tracce (5.565.384 campioni grezzi ridotti a 5.402.386 secondi attivi, con 162.998 righe vuote rimosse ai margini dell'intervento) esclusivamente al termine del flusso di pulizia.

La prima trasformazione eseguita sulla griglia integra consiste nell'identificazione degli outlier fisiologici **prima** di qualsiasi operazione di interpolazione o trascinamento in avanti. Per eliminare il rischio di divergenze tra i moduli Python di pulizia e gli schemi di catalogo del database, i limiti fisiologici di plausibilità sono stati centralizzati in un'unica fonte di verità (**Single Source of Truth**), rappresentata dal dizionario `TRACK_SPECS` nel modulo `src/config.py`: frequenza cardiaca considerata valida tra 20 e 250 bpm; saturazione periferica considerata valida tra il 50% e il 100%; pressione sistolica considerata valida tra 40 e 250 mmHg; pressione diastolica considerata valida tra 10 e 180 mmHg (uniformando il limite superiore tra codice di pulizia e catalogo metadati); pressione media considerata valida tra 20 e 220 mmHg.

I valori osservati che cadono al di fuori di questi intervalli non vengono cancellati distruttivamente dalla riga originale, bensì etichettati tramite cinque flag booleani dedicati (`HR_outlier`, `SPO2_outlier`, `SBP_outlier`, `DBP_outlier`, `MBP_outlier`). Calcolare questi flag sui dati grezzi prima dell'imputazione — e mascherare temporaneamente gli outlier durante il calcolo dell'interpolazione e del forward-fill — risponde a una precisa motivazione clinica e statistica: se gli outlier venissero valutati dopo il trascinamento LOCF, una singola lettura pressoria aberrante causata dalla compressione accidentale del bracciale da parte del chirurgo verrebbe propagata in avanti per i successivi 300 secondi, trasformando un artefatto puntuale di un secondo in 301 secondi consecutivi di falsi outlier e corrompendo le statistiche di qualità.

Successivamente, il modulo applica una strategia differenziata di imputazione (con tracciamento esplicito tramite flag booleani `*_imputed` per ciascuno dei cinque canali) per gestire la disomogeneità fisiologica delle sorgenti sensoristiche: per i segnali fisiologici continui, segnatamente la frequenza cardiaca (`Solar8000/HR`) e la saturazione periferica (`Solar8000/PLETH_SPO2`), le brevi perdite di segnale causate da artefatti di movimento vengono sanate mediante la funzione dedicata `interpolate_short_gaps(series, max_gap=5)`:

$$y(t) = y(t_0) + \frac{y(t_1) - y(t_0)}{t_1 - t_0}(t - t_0)$$

A differenza del metodo standard `.interpolate(limit=5)` della libreria `pandas` — che riempie indiscriminatamente i primi cinque campioni anche all'inizio di un'interruzione strumentale lunga diversi minuti, generando "code" sintetiche prive di significato biologico — la funzione `interpolate_short_gaps` calcola preliminarmente l'ampiezza complessiva di ciascun blocco contiguo di valori mancanti e applica l'interpolazione lineare **esclusivamente** ai buchi interni di durata totale ≤ 5 s compresi tra due misure valide. Le interruzioni di durata superiore a cinque secondi e i valori mancanti periferici vengono lasciati intatti a `NaN`, mentre i punti effettivamente ricostruiti vengono marcati nei flag booleani `HR_imputed` e `SPO2_imputed`; per i segnali pressori non invasivi (`NIBP_SBP`, `NIBP_DBP`, `NIBP_MBP`), la misurazione è intrinsecamente intermittente, avvenendo a intervalli discreti compresi tipicamente tra due e cinque minuti mediante il ciclo oscillometrico del bracciale. In questo scenario viene adottata la tecnica **Last Observation Carried Forward** (LOCF), propagando l'ultimo valore valido non-outlier:

$$y(t) = y(t_{\text{last}}) \quad \forall t \in [t_{\text{last}}, t_{\text{last}} + 300 \text{ s}]$$

La propagazione è limitata a una finestra massima di 300 secondi (cinque minuti) e viene tracciata individualmente per ciascuna componente pressoria tramite le colonne booleane `SBP_imputed`, `DBP_imputed` e `MBP_imputed`. Tracciare separatamente le tre componenti (anziché dedurre il numero di imputazioni indirettamente dalla sola età del bracciale) garantisce conteggi di qualità esatti anche nei rari istanti in cui il monitor restituisce una lettura parziale.

A supporto della trasparenza analitica, dell'auditing clinico e del filtraggio a valle nelle regole diagnostiche, il modulo calcola contestualmente la variabile temporale `nibp_age_s`:

$$\texttt{nibp_age_s} = t - t_{\text{last_cuff}}$$

Tale grandezza quantifica in ogni secondo l'età dell'ultima misurazione pressoria valida registrata su almeno un canale del bracciale. Questo metadato si rivela determinante sia per il clinico sia per il motore di **anomaly detection**: come vedremo nel Capitolo 4, le regole cliniche interrogano direttamente `nibp_age_s` per inibire l'emissione di allarmi emodinamici qualora la pressione propagata per LOCF abbia superato la soglia di freschezza clinica di 120 secondi.

La Figura 2.3 mette a confronto l'andamento del tracciato della frequenza cardiaca di un paziente reale prima e dopo l'intervento di bonifica. Nella parte superiore, il segnale grezzo estratto dal

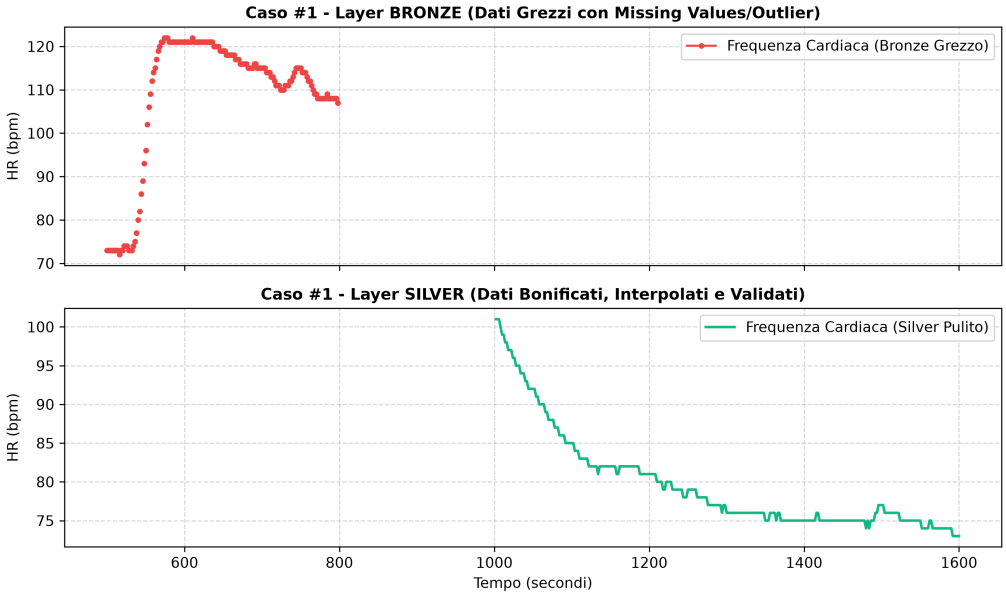


Figura 2.3: Confronto visivo tra il tracciato della frequenza cardiaca grezzo (Bronze) e bonificato (Silver).

livello **Bronze** presenta evidenti salti e interruzioni causati da temporanee perdite di aderenza del sensore; nella parte inferiore, il segnale pulito del livello **Silver** si presenta continuo nei micro-gap fisiologici ma fedele alle reali interruzioni prolungate.

L'implementazione concreta di queste trasformazioni è descritta nel Listato 2.1, estratto dal modulo `src/silver/clean.py`.

```
// Codice estratto da src/silver/clean.py
# 1. Identifica gli outlier fisiologici SUI DATI GREZZI (da config.TRACK_SPECS) prima dell'imputazione
for track_name, flag_col in outlier_map.items():
    spec = config.TRACK_SPECS.get(track_name, {})
    val = df_clean[track_name]
    df_clean[flag_col] = val.notna() & ((val < spec["min"]) | (val > spec["max"]))

# 2. Segnali continui (HR e SpO2): interpola solo buchi interni <= 5s senza propagare outlier
clean_hr, hr_imp = interpolate_short_gaps(raw_hr.mask(out_hr), max_gap=5)
df_clean['Solar8000/HR'] = clean_hr.where(~out_hr, raw_hr)
df_clean['HR_imputed'] = hr_imp & (~out_hr)

# 3. Segnali NIBP: calcolo nibp_age_s e LOCF individuale (max 300s) senza trascinare outlier
last_measure_time = time_series.where(any_valid_nibp).ffill()
df_clean['nibp_age_s'] = (time_series - last_measure_time).fillna(9999.0).astype(float)
for t, o, imp_col in present_nibp:
    filled_col = df_clean[t].mask(df_clean[o]).ffill(limit=300)
    df_clean[t] = filled_col.where(~df_clean[o], df_clean[t])
    df_clean[imp_col] = was_na & df_clean[t].notna()

# 4. Scarta solo alla fine le righe in cui tutte le tracce vitali sono simultaneamente nulle
df_clean = df_clean.dropna(subset=vital_cols, how='all').copy()
```

Listing 2.1: Algoritmo di identificazione preventiva degli outlier, interpolazione selettiva dei gap brevi e tracciamento esplicito nel layer Silver.

Sintesi di data governance nel layer Silver

L'architettura del layer Silver garantisce che nessun artefatto venga propagato o nascosto: i limiti fisiologici sono letti da un'unica fonte centralizzata (`TRACK_SPECS`), gli outlier sono marcati prima dell'imputazione così da non essere trascinati dal forward-fill, l'interpolazione lineare è circoscritta ai soli buchi interni ≤ 5 s (`interpolate_short_gaps`), e ciascuno dei 5 canali dispone di un flag booleano dedicato (`*_imputed`) oltre al contatore di freschezza `nibp_age_s`.

Infine, i dati bonificati vengono salvati in formato Parquet all'interno di una gerarchia di directory partizionata fisicamente per reparto chirurgico di appartenenza (`department`):

`data/silver/department=<REPARTO>/case_<ID>.parquet`

Questa organizzazione ricalca il principio del **partition pruning** dei moderni **data lakehouse**: quando un'interrogazione o una procedura di addestramento richiede i dati relativi a una specifica specialità chirurgica, il lettore Apache Parquet ispeziona unicamente il sottoalbero della directory corrispondente senza scansionare l'intero archivio su disco, abbattendo le operazioni di I/O e velocizzando l'elaborazione.

2.4 Evoluzione del modello dati da tabella piatta a documento BSON

L'architettura della pipeline non si limita a innalzare la qualità intrinseca dei valori numerici, ma trasforma radicalmente il modello concettuale del dato nel passaggio verso il livello **Gold**.

Nei livelli **Bronze** e **Silver**, i dati risiedono sotto forma di tabelle piatte bidimensionali all'interno di file Parquet (6 colonne nel **Bronze**, arricchite a 18 colonne nel **Silver** per includere i dieci flag booleani di outlier e imputazione, l'età della misura pressoria `nibp_age_s` e l'identificativo `case_id`). Questo modello tabulare classico, pur risultando ideale per elaborazioni vettoriali in memoria tramite librerie quali `pandas`, mostra evidenti limiti nel contesto dell'archiviazione distribuita e del monitoraggio clinico in tempo reale: ogni riga ripete pedissequamente gli identificatori del paziente, alloca colonne booleane anche quando tutti i flag sono falsi e richiede schemi rigidi poco adatti all'evoluzione dei parametri monitorati.

Nel livello **Gold**, il dato viene rimodellato come documento gerarchico BSON all'interno di MongoDB. La struttura del documento, mostrata nel Listato 2.2, è articolata in quattro sezioni logiche: il campo radice `timestamp`, che definisce l'ordinamento temporale ed è memorizzato come oggetto nativo `ISODate`; il sotto-documento `metadata`, che raccoglie in modo compatto tutte le proprietà immutabili del paziente e del sensore, tra cui l'identificativo del caso (`case_id`), il nome del monitor sorgente (`sensor_name`), il reparto chirurgico, l'età e il sesso; il sotto-documento `metrics`, che incapsula i soli valori numerici delle misurazioni fisiologiche effettivamente presenti in quel secondo, opportunamente sanitizzati sostituendo i caratteri di barra con trattini bassi per evitare conflitti con la sintassi a percorsi di MongoDB; il sotto-documento `quality_flags`, che eredita dal livello **Silver** sia i cinque flag booleani `*_outlier` sia i cinque flag `*_imputed` e il contatore di freschezza `nibp_age_s`, preservando integralmente la qualifica di affidabilità e provenienza di ciascuna misurazione.

```
// Documento gerarchico BSON archiviato nella collezione 'vital_signals'
{
```

```
  "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
  "timestamp": ISODate("2020-01-01T08:15:30Z"),
  "metadata": {
    "case_id": 1,
    "sensor_name": "Solar8000",
    "department": "General surgery",
    "age": 64,
    "sex": "M"
  },
  "metrics": {
    "Solar8000_HR": 72.0,
    "Solar8000_PLETH_SPO2": 98.0,
    "Solar8000_NIBP_SBP": 120.0,
    "Solar8000_NIBP_DBP": 80.0,
    "Solar8000_NIBP_MBP": 93.3
  },
  "quality_flags": {
    "HR_outlier": false, "SPO2_outlier": false,
    "SBP_outlier": false, "DBP_outlier": false, "MBP_outlier": false,
    "HR_imputed": false, "SPO2_imputed": false,
    "SBP_imputed": true, "DBP_imputed": true, "MBP_imputed": true,
    "nibp_age_s": 14.0
  }
}
```

Listing 2.2: Evoluzione del dato dal formato Parquet piatto al documento BSON nel layer Gold.

Questa rappresentazione semi-strutturata si rivela decisiva per sfruttare appieno le capacità del motore **Time Series** di MongoDB, poiché isola i metadati descrittivi dal flusso continuo delle metriche numeriche, abilitando strategie avanzate di compressione e indicizzazione.

CAPITOLO 3

Archiviazione e governance con MongoDB Time Series

3.1 Layer Gold e archiviazione con MongoDB Time Series

Il livello **Gold** rappresenta lo strato di vertice dell'architettura dati, destinato ad alimentare in modo tempestivo e strutturato le interrogazioni analitiche, l'interfaccia grafica utente e gli algoritmi di rilevamento delle anomalie. I dati bonificati nel livello **Silver** vengono convogliati verso una base di dati MongoDB versione 8.0, all'interno della quale è configurata una collezione temporale nativa denominata `vital_signals`.

La funzionalità **Time Series Collection**, introdotta nelle recenti versioni di MongoDB, costituisce un'innovazione architetturale di rilievo rispetto alle collezioni a documenti tradizionali. In un database a documenti convenzionale, la memorizzazione di milioni di misurazioni ad alta frequenza comporterebbe un overhead insostenibile: ogni singolo secondo richiederebbe un documento BSON completo con il rispettivo identificatore univoco `_id`, le chiavi di campo replicate e le strutture di indice correlate. Nelle collezioni temporali, invece, MongoDB astrae la gestione fisica del dato raggruppando in modo del tutto trasparente le osservazioni temporali contigue all'interno di blocchi compressi denominati **bucket**.

La creazione della collezione `vital_signals`, implementata nello script `load_mongo.py`, richiede la definizione esplicita di tre parametri architetturali: il parametro `timeField`, impostato sul campo radice `timestamp`, che indica al motore il campo di riferimento temporale per ordinare e partizionare i record; il parametro `metaField`, impostato sul sotto-documento `metadata`, che raccoglie gli attributi descrittivi immutabili della serie associata a quel paziente; il parametro `granularity`, impostato al valore `seconds`, che suggerisce al motore la frequenza attesa delle misurazioni per ottimizzare la dimensione e la durata temporale di ciascun **bucket**.

All'interno dell'infrastruttura di archiviazione, MongoDB organizza i dati nella collezione di sistema interna `system.buckets.vital_signals`. In tale collezione, ogni singolo documento BSON non rappresenta una singola misurazione istantanea, bensì un intero **bucket** che racchiude un blocco compresso di campioni temporali consecutivi. Dal punto di vista strutturale, il documento di **bucket** si articola in tre componenti fondamentali: l'identificativo univoco `_id`, il sotto-documento `control` e il contenitore binario `data`. Il sotto-documento `control` ospita i metadati di controllo dell'intervallo, tra cui i valori estremi del timestamp (`min.timestamp` e `max.timestamp`), i valori minimi e massimi assunti dalle metriche numeriche all'interno del blocco, la versione dello schema interno (`version: 1`) e il contatore dei campioni (`count`), che può accogliere fino a 1.000 osservazioni temporali o estendersi fino al limite orario previsto dalla granularità definita.

Quando giungono nuovi campioni recanti i medesimi attributi immutabili (ossia lo stesso paziente e monitor), il motore inserisce i dati nel **bucket** aperto corrente fino al raggiungimento della capacità massima. Per comprimere le sequenze temporali all'interno della sezione `data`, MongoDB adotta una combinazione sinergica di algoritmi colonnari avanzati: l'asse temporale viene compresso mediante l'algoritmo **Delta-of-Delta**, originariamente introdotto nell'architettura Gorilla di Facebook per le serie storiche ad altissima frequenza. Indicando con t_i il timestamp dell'osservazione corrente, la procedura calcola la differenza temporale di primo ordine rispetto

al campione precedente, $D_i = t_i - t_{i-1}$, e successivamente la variazione di secondo ordine rispetto all'incremento antecedente, $\Delta_i = D_i - D_{i-1}$. Nel contesto del monitoraggio intraoperatorio ad alta fedeltà a 1 Hz, i campioni si susseguono a intervalli rigorosamente costanti di un secondo: ne consegue che $D_i = 1$ s per ogni istante e, di riflesso, la derivata seconda temporale risulta identicamente nulla, $\Delta_i = 0$. In virtù delle regole di codifica bit-packing dell'algoritmo, il valore nullo viene serializzato impiegando un singolo bit `0`, invece degli 8 byte (64 bit) ordinariamente necessari per rappresentare un timestamp Unix standard a 64 bit o una data BSON. L'ingombro dell'ordinamento temporale crolla quindi a frazioni trascurabili di memoria; i vettori numerici delle metriche biometriche continue (quali frequenza cardiaca e saturazione) vengono codificati sfruttando la correlazione tra campioni consecutivi tramite differenziazione binaria XOR sulle mantisse in virgola mobile a 64 bit (standard IEEE 754), seguita da una compressione a blocchi con l'algoritmo **Zstandard** (zstd).

Un ulteriore beneficio architetturale di MongoDB rispetto al formato Parquet risiede nella gestione della sparsità e dei valori mancanti. Nel formato Parquet, ogni colonna numerica della tabella partizionata alloca necessariamente una maschera di bit di definizione (**definition level bitmask**) per tracciare per ciascuna riga la presenza o assenza del dato. Al contrario, all'interno della struttura flessibile BSON adottata da MongoDB, se un parametro vitale non viene misurato in un determinato secondo (come avviene tipicamente per la pressione non invasiva nei lunghi intervalli di riposo tra due cicli di gonfiaggio del bracciale), il relativo campo viene semplicemente omesso dal sotto-documento `metrics`. Questa assenza non comporta alcuna allocazione di byte né padding numerico, massimizzando la densità informativa dello storage.

3.2 Benchmark prestazionale e di occupazione dello storage

Per valutare in modo quantitativo e scientificamente rigoroso i benefici apportati dalla transizione dai file Parquet alla collezione temporale di MongoDB, è stata eseguita una campagna di benchmark automatizzata mediante lo script `src/analysis/benchmark_etl.py`. Al fine di garantire che il confronto avvenga esattamente sullo stesso perimetro sperimentale certificato dai report di qualità, lo script acquisisce dal catalogo e dai file di stato l'identificativo dell'esecuzione (`run_id`) e la versione della pipeline (`pipeline_version = "1.1.0"`), salvandoli nell'intestazione di `data/benchmark_report.json`.

La misurazione ha preso in esame l'intera coorte di **500 casi clinici completi**, per un totale di **5.402.386 punti temporali** bonificati nel livello **Gold** (a fronte di 5.565.384 campioni grezzi originari nel livello **Bronze**). La sperimentazione ha indagato tre parametri cardine: l'occupazione fisica dello storage su disco (distinguendo rigorosamente i dati compressi dagli indici), la latenza di scansione delle interrogazioni temporali a parità di carico di lavoro (**like-for-like**) e la completezza logica e fisica delle celle numeriche. I risultati ottenuti sono compendiati nella Tabella 3.1 e illustrati graficamente nella Figura 3.1.

Parametro / Metrica	Bronze (grezzo, 6 col.)	Silver (arricchito, 18 col.)	Gold (MongoDB 8.0 TS)
Occupazione disco (500 casi)	33.95 MB	44.52 MB (+31.1% con flag DQ)	**15.60 MB** (-54.1% vs Bronze / -65.0% vs Silver)
Overhead indici B-Tree (`totalIndexSize`)	N/A (File Parquet)	N/A (File Parquet)	+2.56 MB (Totale `collStats`: 18.16 MB)
Latenza scansione singolo caso	26.95 ms (Parquet)	26.95 ms (Parquet)	**12.02 ms** (Pushdown indicizzato)
Fattore di velocizzazione	1.0× (Riferimento)	1.0×	**2.2× più veloce**
Valori mancanti (Null Rate logico / fisico)	66.3%	26.3% (post-LOCF)	26.3% logico / **0 byte fisici** (Omissione BSON)

Tabella 3.1: Confronto quantitativo di storage, latenza e qualità dati tra i layer su 500 casi clinici (5.402.386 record Gold).

L'analisi dei volumi di archiviazione riportati nella Tabella 3.1 mette in luce una dinamica architetturale particolarmente istruttiva della **Medallion Architecture**:

- **Espansione informativa nel livello Silver (+31.1%):** il passaggio dai file Parquet grez-

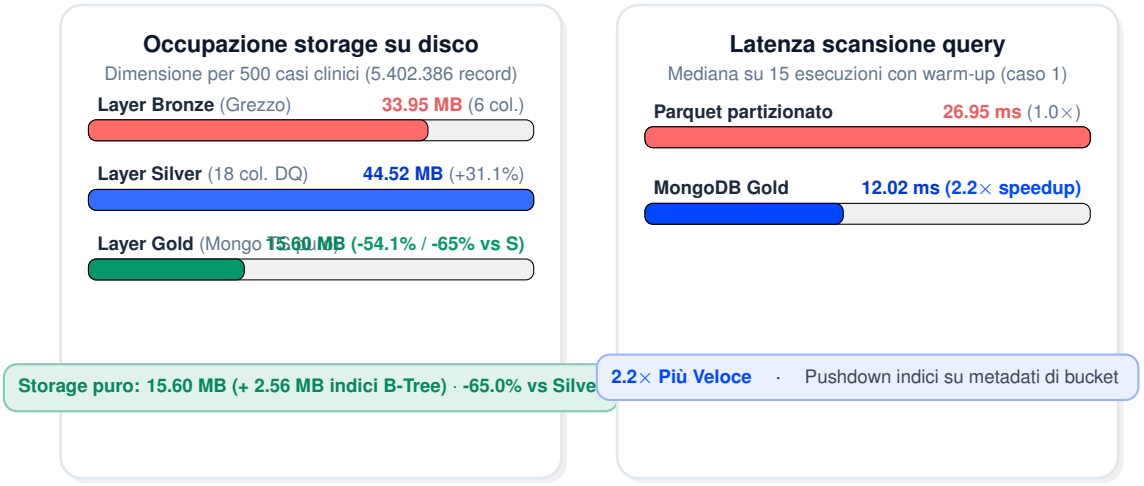


Figura 3.1: Benchmark comparativo dell'occupazione disco e della latenza di query sui 500 casi clinici generato in TikZ.

zi del livello **Bronze** (33.95 MB) al livello **Silver** (44.52 MB) comporta un incremento di circa 10.57 MB. Questo fenomeno è la conseguenza diretta della natura del layer **Silver**, che non svolge una funzione di compressione bensì di **arricchimento e data governance**. Mentre un file Bronze contiene solamente 6 colonne per lo più popolate da valori `NaN` (66.3% di sparsità, che l'algoritmo Snappy/RLE di Parquet comprime con estrema facilità), il file Silver espande la struttura tabellare a **18 colonne** (aggiungendo 5 flag booleani `*_outlier`, 5 flag booleani `*_imputed`, l'identificativo `case_id` e il contatore `nibp_age_s`) e densifica le serie pressorie tramite **forward-fill** (riducendo le celle vuote dal 66.3% al 26.3%). In particolare, la colonna `nibp_age_s`, memorizzata come `float64` e incrementata di un'unità a ogni secondo $(0, 1, 2, \ldots, 300)$, introduce un'entropia continua che impedisce al formato colonnare Parquet di applicare il **Run-Length Encoding** (RLE) sulle righe adiacenti;

- **Compressione strutturale nel livello Gold (-65.0% rispetto a Silver, -54.1% rispetto a Bronze):** la successiva promozione verso la collezione temporale nativa di MongoDB 8.0 abbatte l'occupazione fisica dei dati (`storageSize` puro dei **bucket** compressi) a soli **15.60 MB**, conseguendo una riduzione del 65.0% rispetto al layer Silver arricchito e del 54.1% rispetto al layer Bronze grezzo. Per garantire il massimo rigore metodologico, lo script `benchmark_etl.py` interroga il comando `collStats` di MongoDB scorporando esplicitamente lo spazio occupato dai dati compressi (`storageSize = 15.60 MB`) dall'ingombro delle strutture di indicizzazione B-Tree su `metadata.case_id` e `timestamp` (`totalIndexSize = 2.56 MB`, per un'allocazione complessiva di 18.16 MB). In questo modo si evita di confondere il peso delle strutture di accesso rapido con la dimensione effettiva delle serie temporali compresse tramite **Delta-of-Delta** e **Zstandard**.

Un'analoga trasparenza metodologica è stata adottata per la misurazione del tasso di valori mancanti (**Null Rate**). In una prima versione preliminare del benchmark, il tasso di valori nulli nel livello **Gold** veniva riportato convenzionalmente pari a `0.0%` in virtù dell'omissione fisica dei campi nulli nei documenti BSON. Per evitare che tale convenzione potesse apparire come una sovrastima della copertura reale dei sensori, nello script `benchmark_etl.py` è stata introdotta la funzione `calculate_mongo_null_rate(db)`, che interroga i documenti appartenenti ai casi con stato `committed` verificando, tramite l'operatore `$exists: True`, l'effettiva presenza di ciascuna delle cinque tracce biometriche rispetto al totale teorico di celle (documenti totali $\times$ 5). Il calcolo restituisce un tasso di sparsità logica pari al **26.3%**, identico a quello misurato sul layer **Silver** post-LOCF: ciò dimostra matematicamente che non vi è alcuna perdita né alterazione informa-

tiva nel passaggio da Silver a Gold, pur confermando il vantaggio fisico dell'architettura BSON che alloca **0 byte su disco** per quel 26.3% di misurazioni assenti.

Sul fronte delle prestazioni computazionali, il protocollo di test ha adottato una metodologia comparativa rigorosamente simmetrica (**like-for-like**): è stata misurata la mediana dei tempi di risposta su quindici iterazioni successive relative all'aggregazione sul singolo caso clinico (`case_id = 1`), precedute da una fase di riscaldamento (**warmup**) per stabilizzare le cache del sistema operativo e del motore WiredTiger. L'interrogazione mirava a calcolare la media della frequenza cardiaca per l'intero arco temporale dell'intervento. La lettura e aggregazione del file Parquet partizionato ha richiesto una latenza mediana di **26.95 ms**, mentre la corrispondente pipeline di aggregazione in MongoDB (`$match` sul metadato indicizzato seguito da `$group`) è stata evasa in **12.02 ms**. Il motore di MongoDB consegue dunque un fattore di velocizzazione pari a **2.2 volte**, beneficiando del filtraggio anticipato (**query pushdown**) a livello di metadati del **bucket** e della scansione selettiva dei soli blocchi temporali pertinenti.

Vantaggi architetturali di MongoDB Time Series su larga scala

Su una coorte di 500 interventi chirurgici (5.4 milioni di record), la combinazione della compressione a **bucket** (**Delta-of-Delta** + **Zstandard**) con l'omissione BSON dei campi nulli abbatte lo storage puro a **15.60 MB** (-65.0% rispetto al layer Silver arricchito e -54.1% rispetto al Bronze). Contestualmente, il **query pushdown** sui metadati indicizzati del **bucket** dimezza i tempi di risposta analitica sul singolo paziente (**12.02 ms** contro **26.95 ms**, speedup di **2.2**×).

3.3 Data governance artigianale in ottica data lakehouse

Un aspetto centrale del progetto, concordato e approfondito durante le fasi di revisione con la relatrice, risiede nella formalizzazione della **data governance**. Nei moderni contesti industriali, l'adozione di un'architettura a livelli richiede meccanismi stringenti per garantire l'integrità strutturale, il tracciamento della provenienza end-to-end e l'affidabilità delle scritture anche a fronte di interruzioni impreviste. Piattaforme dedicate quali Delta Lake, Apache Iceberg o Apache Hudi forniscono queste garanzie mediante registri di transazioni distribuiti (**transaction log**) e cataloghi integrati all'interno di motori di calcolo analitico quali Spark o Trino.

Nel presente lavoro, per mantenere l'architettura compatta, autosufficiente ed eseguibile in ambienti ospedalieri locali senza dipendenze computazionali onerose, si è scelto di emulare artigianalmente le proprietà fondamentali di un **data lakehouse** sfruttando esclusivamente gli strumenti e le primitive native di MongoDB unitamente a protocolli applicativi di controllo in Python. Tale impostazione poggia su quattro componenti complementari: il catalogo di registrazione a macchina a stati, la sincronizzazione centralizzata dei metadati descrittivi, la validazione formale dei contratti dati e un protocollo di pubblicazione a due fasi con recupero automatico.

3.3.1 Catalogo, macchina a stati e provenienza (collezione registry)

Per tenere traccia dello stato di ingestione di ciascun caso clinico e garantire la ricostruibilità storica del processo di caricamento, è stata istituita la collezione `registry`. Tale collezione funge da catalogo centrale e opera come l'equivalente di un metastore applicativo e di un registro di stato delle transazioni.

Al fine di collegare in modo inequivocabile i documenti caricati nel livello **Gold** con i file del livello **Silver** e i report di qualità, il documento di catalogo è stato esteso per registrare non solo i conteggi, ma anche lo stato del ciclo di vita del caricamento e i metadati completi di **lineage**. Come mostrato nel Listato 3.1, ogni voce del registro certifica:

- l'identificativo univoco del caso clinico (`case_id`) e la versione formale dello schema logico (`schema_version`);
- lo stato transazionale del caso (`status`), vincolato ai tre valori ammissibili `"loading"`, `"committed"` e `"failed"`, che consente a tutti i moduli a valle di distinguere i caricamenti completati da quelli in corso o interrotti;
- il numero complessivo di punti temporali validati e immessi nella serie (`record_count`);
- il sotto-documento `provenance`, che registra la fase di elaborazione (`silver_to_gold`), il timestamp UTC, l'identificativo univoco dell'esecuzione (`run_id`, ereditato dal report Silver per garantire la tracciabilità trasversale), la versione del codice della pipeline (`pipeline_version = "1.1.0"`) e il dizionario `cleaning_params` con gli iperparametri esatti applicati durante la bonifica (`interpolation_max_gap_s: 5` e `nibp_ffill_limit_s: 300`).

```
// Validazione applicata alla collezione 'registry' in MongoDB (mongo/
    registry_schema.js)
db.runCommand({
  collMod: "registry",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["case_id", "schema_version", "status", "provenance"],
      properties: {
        case_id: {
          bsonType: ["int", "long"],
          description: "ID del caso clinico, intero a 32 o 64 bit obbligatorio"
        },
        schema_version: {
          bsonType: "string",
          description: "Versione dello schema logico del documento"
        },
        status: {
          enum: ["loading", "committed", "failed"],
          description: "Stato transazionale del caricamento Gold (Two-Phase
    Load)"
        },
        record_count: {
          bsonType: ["int", "long"],
          description: "Numero complessivo di record temporali pubblicati"
        },
        provenance: {
          bsonType: "object",
          required: ["step", "timestamp"],
          properties: {
            step: { bsonType: "string" },
            timestamp: { bsonType: "date" },
            run_id: { bsonType: "string" },
            pipeline_version: { bsonType: "string" },
            cleaning_params: { bsonType: "object" }
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
```

```
41 });
```

Listing 3.1: Schema formale di validazione JSON Schema per la collezione registry con stato e lineage.

La collezione `registry` è protetta da regole native di **JSON Schema Validation** configurate con livello rigoroso (`validationLevel: "strict"`) e azione di rifiuto immediato (`validationAction: "error"`). In questo modo, qualsiasi tentativo di inserire nel catalogo record privi dello stato transazionale o dei vincoli di provenienza viene categoricamente respinto dal motore del database.

3.3.2 Metadati descrittivi delle tracce e sincronizzazione automatica (track_metadata)

Accanto al catalogo delle esecuzioni, la governance necessita di una definizione semantica chiara del significato di ciascuna variabile misurata. A questo scopo è stata creata la collezione ausiliaria `track_metadata`, inizializzata al primo avvio del container tramite lo script `mongo/registry_schema.js`. Per ciascuna metrica fisiologica registrata dal monitor `Solar8000`, la collezione documenta il nome formale della traccia, la versione del contratto dati, l'unità di misura fisica adottata (`bpm`, percentuale o `mmHg`) e l'intervallo numerico fisiologicamente plausibile (`expected_range`).

Tuttavia, poiché lo script `/docker-entrypoint-initdb.d/registry_schema.js` viene eseguito da Docker esclusivamente alla prima creazione fisica del volume dati, un'eventuale modifica successiva delle soglie cliniche nel codice Python rischierebbe di disallineare il catalogo MongoDB rispetto alle regole effettivamente applicate dalla pipeline. Per eliminare alla radice questo rischio di deriva (**configuration drift**), nel modulo `src/gold/load_mongo.py` è stata implementata la funzione `sync_track_metadata(db)`: a ogni esecuzione del caricamento Gold, la procedura legge il dizionario centralizzato `config.TRACK_SPECS` (unica fonte di verità del progetto) ed esegue un'operazione idempotente di `update_one(..., upsert=True)` sulla collezione `track_metadata`, garantendo che il contratto esposto nel database rifletta sempre fedelmente i parametri attivi nel codice.

3.3.3 Limiti delle Time Series, protocollo a due fasi e validazione applicativa

Un'analisi rigorosa della piattaforma impone di evidenziare con trasparenza due vincoli architetturali intrinseci delle **Time Series Collection** in MongoDB 8.0 e le soluzioni ingegneristiche adottate per superarli:

1. **Assenza di JSON Schema Validator nativo sulle collezioni Time Series:** MongoDB non consente di associare regole di validazione `$jsonSchema` direttamente a una collezione temporale (poiché le scritture vengono trasformate internamente in operazioni di compressione sui documenti di `system.buckets`). Per garantire comunque il rispetto formale del contratto dati prima dell'ingresso nel layer **Gold**, in `load_mongo.py` è stata introdotta la funzione di validazione applicativa `validate_records(records, case_id)`. Prima di invocare `insert_many`, la funzione ispeziona il batch di documenti verificando la presenza dei campi obbligatori (`timestamp`, `metadata.case_id`) e la conformità di tutte le misure presenti in `metrics` rispetto ai domini fisiologici `config.TRACK_SPECS`; in presenza di una sola violazione, il caricamento del caso viene bloccato preventivamente sollevando un'eccezione `ValueError`;
2. **Assenza di transazioni multi-documento distribuite:** le **Time Series Collection** non supportano operazioni di scrittura all'interno di sessioni transazionali ACID multi-documento (sollevando l'errore nativo 263, `OperationNotSupportedInTransaction`). Di conseguenza, se un processo di caricamento si interrompesse a metà dell'inserimento dei blocchi di un paziente, nella collezione `vital_signals` rimarrebbero frammenti parziali di serie temporale incoerenti con il catalogo `registry`.

Per garantire l'atomicità logica della pubblicazione e l'isolamento dei lettori a valle senza ricorrere

a transazioni native, in `src/gold/load_mongo.py` è stato progettato un protocollo di **caricamento a due fasi (*Two-Phase Load*) con recupero automatico**:

- **Fase 0 – Bonifica automatica di avvio (`recover_incomplete_loads`):** prima di iniziare l'elaborazione della coorte, il caricatore ispeziona la collezione `registry` alla ricerca di eventuali casi rimasti in stato `"loading"`, `"failed"` o privi dell'attributo `status` a causa di crash o interruzioni forzate di esecuzioni precedenti. Per ciascuno di essi, la procedura rimuove automaticamente i punti parziali presenti in `vital_signals` ed elimina la voce incompleta dal `registry`, ripristinando uno stato iniziale pulito;
- **Fase 1 – Prenotazione (`status: "loading"`) e pulizia preventiva:** per ogni caso da caricare (ossia non ancora presente nel catalogo con stato `"committed"`), lo script registra immediatamente nel `registry` un documento con `status: "loading"` e `record_count: 0`, seguito da una cancellazione preventiva `delete_many({"metadata.case_id": case_id})` su `vital_signals` e dalla validazione formale `validate_records`;
- **Fase 2 – Inserimento a blocchi e Commit (`status: "committed"`) o Rollback (`status: "failed"`):** i documenti BSON vengono inseriti in `vital_signals` a blocchi di 10.000 unità. Soltanto al termine dell'ultimo blocco, il documento nel `registry` viene promosso allo stato finale `status: "committed"` con il conteggio esatto dei record e il `run_id` corrente. Qualora si verifichi un qualsiasi errore di rete, memoria o validazione durante l'inserimento, il blocco `except` intercetta l'eccezione, marca il caso come `status: "failed"` nel catalogo ed esegue immediatamente una `delete_many` compensativa su `vital_signals`, impedendo la sopravvivenza di dati orfani;
- **Isolamento coerente dei lettori (*Read Committed*):** tutti i moduli di interrogazione e analisi a valle — la funzione `list_loaded_cases()` esposta dall'API REST, la funzione `load_from_gold()` in `detector.py`, lo script di valutazione `evaluate_models.py` e il benchmark `benchmark_etl.py` — interrogano la collezione `registry` applicando rigorosamente il filtro `{"status": "committed"}`. In tal modo, nessun caso parziale o in fase di caricamento può mai essere letto dalla dashboard o inquinare l'addestramento dei modelli di **machine learning**.

3.4 Pipeline di aggregazione analitica e ottimizzazione delle query

La disponibilità dei dati ad alta frequenza nel livello **Gold** consente di sviluppare pipeline di interrogazione analitica avanzate per alimentare in tempo reale la visualizzazione grafica e gli algoritmi diagnostici. Per evitare che la complessità computazionale si trasferisca sull'applicazione web o sull'infrastruttura di rete, le elaborazioni intensive di filtraggio, raggruppamento e aggregazione temporale vengono demandate direttamente al motore interno di MongoDB, sfruttando il framework di aggregazione nativo (**Aggregation Pipeline**).

3.4.1 Downsampling temporale efficiente lato database con $dateTrunc

Nel monitoraggio intraoperatorio continuo, un singolo intervento chirurgico di media o lunga durata può protrarsi per diverse ore, accumulando decine di migliaia di punti temporali per paziente (oltre 21.000 campioni per una procedura di sei ore). Il trasferimento indiscriminato dell'intero tracciato a frequenza di 1 Hz dal database al browser del client comporterebbe un sovraccarico insostenibile: il payload JSON della risposta HTTP supererebbe decine di megabyte, saturando la banda di rete e rallentando il motore di rendering grafico JavaScript dell'interfaccia utente.

Per risolvere questa criticità, nel modulo `src/analysis/queries.py` è stata sviluppata la funzione `downsample_case`, richiamabile dall'API specificando il parametro `window_seconds`. La funzione esegue una riduzione controllata della risoluzione temporale (**downsampling**) interamente a livello di base di dati mediante una pipeline strutturata su tre stadi consecutivi, come illustrato nel Li-

stato 3.2: lo stadio `$match` seleziona esclusivamente i documenti pertinenti al paziente richiesto sfruttando il campo indicizzato `metadata.case_id`; lo stadio `$group` impiega l'operatore temporale `$dateTrunc` per troncare e raggruppare i timestamp delle misurazioni in intervalli discreti di ampiezza prefissata (ad esempio 30, 60 o 300 secondi), specificando l'unità di misura temporale (`unit: "second"`) e la dimensione del contenitore (`binSize: window_seconds`). Contestualmente, per ciascuna metrica vitale presente nel sotto-documento `metrics`, il gruppo calcola la media aritmetica dei valori osservati nella finestra mediante l'accumulatore `$avg`; lo stadio `$sort` ordina cronologicamente le finestre aggregate garantendo la continuità sequenziale del flusso restituito.

```
// Aggregazione per il downsampling temporale in MongoDB
db.vital_signals.aggregate([
  // 1. Filtraggio per caso clinico (sfrutta l'indice sui metadati)
  { $match: { "metadata.case_id": case_id } },

  // 2. Raggruppamento temporale a finestre fisse con media delle metriche
  {
    $group: {
      _id: {
        $dateTrunc: {
          date: "$timestamp",
          unit: "second",
          binSize: NumberInt(window_seconds)
        }
      },
      HR:       { $avg: "$metrics.Solar8000_HR" },
      PLETH_SPO2: { $avg: "$metrics.Solar8000_PLETH_SPO2" },
      NIBP_SBP: { $avg: "$metrics.Solar8000_NIBP_SBP" },
      NIBP_DBP: { $avg: "$metrics.Solar8000_NIBP_DBP" },
      NIBP_MBP: { $avg: "$metrics.Solar8000_NIBP_MBP" }
    }
  },

  // 3. Ordinamento cronologico delle finestre aggregate
  { $sort: { _id: 1 } }
]);
```

Listing 3.2: Pipeline di aggregazione MongoDB per il downsampling temporale con l'operatore $dateTrunc.

Questa strategia garantisce che il volume dei dati trasmessi sulla rete si riduca di un fattore esattamente proporzionale all'ampiezza della finestra scelta (ad esempio una contrazione di 60 volte per finestre da un minuto), consentendo alla dashboard web di caricare e visualizzare tracciati chirurgici di svariate ore con latenze inferiori a cento millisecondi.

3.4.2 Ottimizzazione dell'accesso anagrafico ed eliminazione del problema N+1

Un'ulteriore ottimizzazione architetturale riguarda l'estrazione dell'elenco dei pazienti per la barra di navigazione della dashboard, implementata nella rotta REST `GET /cases`.

In un disegno non ottimizzato, l'applicazione interrogherebbe preliminarmente la collezione `registry` per recuperare l'elenco dei casi censiti, per poi iterare su ciascuno di essi ed eseguire una query distinta (`find_one`) sulla collezione `vital_signals` allo scopo di recuperare i metadati di contesto del paziente (reparto chirurgico, età e sesso). Questo approccio ingenuo genera il noto problema delle $N + 1$ query: per una coorte di N pazienti (nel nostro caso $N = 500$), il sistema invierebbe al database 501 interrogazioni separate, moltiplicando i tempi di latenza di rete e il carico di parsing del motore.

Nel presente sistema, il problema è stato risolto strutturando una procedura a due sole operazioni indipendenti dal numero di pazienti della coorte: la prima lettura interroga in blocco la collezione `registry` tramite la funzione `list_loaded_cases(db)`, che filtra esclusivamente i record con stato `{"status": "committed"}` per prelevare i conteggi certificati; la seconda operazione invia a `vital_signals` una pipeline di aggregazione con un singolo stadio `$group` raggruppato per `$metadata.case_id`, che estrae istantaneamente il primo documento associato a ciascun paziente mediante l'operatore `$first` sui campi `department`, `age` e `sex`.

Le due strutture informative vengono poi unite in memoria all'interno del backend Python in un singolo dizionario. In questo modo il numero di interazioni con il database rimane rigorosamente fisso a due, azzerando qualsiasi sovraccarico anche su centinaia di pazienti.

3.5 Uso dei metadati per la data quality e analisi critica

Nel disegno dell'architettura, i metadati svolgono una duplice funzione: abilitano prestazioni di accesso ottimali e governano l'osservabilità della qualità del dato lungo tutte le fasi della pipeline.

Il campo `metaField` della collezione temporale svolge un ruolo tecnologico primario: raccogliendo in modo strutturato l'identificativo del caso, il monitor di provenienza, il reparto chirurgico, l'età e il sesso del paziente, MongoDB genera automaticamente indici compositi che accoppiano i metadati al timestamp. Le interrogazioni analitiche che filtrano per paziente o per reparto beneficiano del cosiddetto **query pushdown**: il motore individua ed esamina esclusivamente i **bucket** pertinenti senza dover decodificare le serie storiche dei pazienti estranei alla query.

Inoltre, per garantire che le valutazioni di qualità effettuate nel livello **Silver** non vadano disperse nella successiva fase di analisi o addestramento dei modelli, ciascun punto memorizzato nel livello **Gold** incorpora l'oggetto esteso `quality_flags`, che racchiude i 5 flag booleani di anomalia puntuale (`*_outlier`), i 5 flag booleani di imputazione sintetica (`*_imputed`) e l'età in secondi della misura pressoria (`nibp_age_s`). Questa architettura realizza di fatto un **Quality Gate granulare a livello di singolo campione (*Row-Level Quality Gate*)**: anziché scartare in blocco un intero intervento chirurgico qualora un sensore presenti una copertura ridotta o si disconnetta temporaneamente durante una manovra anestesiologica — scelta che comporterebbe la perdita di ore di telemetria valida sugli altri canali — la pipeline conserva il caso clinico ma fornisce ai moduli analitici gli attributi necessari per mascherare puntualmente gli artefatti e disabilitare le regole cliniche quando una misura trascinata diventa obsoleta.

A supporto della tracciabilità complessiva, la conclusione della fase di pulizia **Silver** comporta la generazione automatica del file `data/quality_report.json` (accompagnato dall'archiviazione storica in `data/quality_history/` e marcato con `run_id` e `pipeline_version = "1.1.0"`). Per evitare incoerenze matematiche nei conteggi di copertura — che si verificherebbero se i valori osservati venissero contati sulle righe grezze **Bronze** mentre i valori mancanti venissero calcolati dopo lo scarto delle righe vuote — la sezione `sensor_metrics` del report adotta come denominatore comune per tutti gli indicatori il perimetro bonificato `rows_cleaned`. Per ciascuno dei cinque canali fisiologici vale così l'identità esatta:

$$\texttt{observed} + \texttt{imputed} + \texttt{missing} \equiv \texttt{rows_cleaned}$$

A partire da questi conteggi omogenei, il report espone tre percentuali complementari:

- `observed_pct`: la quota di campioni realmente misurati dal sensore hardware sul totale delle righe attive (`observed / rows_cleaned`);
- `coverage_pct`: la copertura finale disponibile dopo l'intervento dell'interpolazione e del **forward-fill** (`(observed + imputed) / rows_cleaned`);

- `temporal_coverage_bronze_pct`: la copertura storica grezza rispetto all'intera durata del file Bronze (`observed_raw_bronze / rows_original`), mantenuta a fini di audit comparativo.

A fronte dei risultati ottenuti sull'intera coorte di 500 casi clinici, è doveroso svolgere tre considerazioni critiche sulla semantica delle metriche rilevate:

1. **Distinzione tra sparsità logica e allocazione fisica nel livello Gold:** come evidenziato nella Tabella 3.1, il livello **Gold** conserva esattamente la medesima sparsità logica del livello **Silver** post-LOCF (**26.3%** di celle mancanti, misurate tramite query `$exists` sulle cinque tracce vitali), ma azzera completamente lo spazio fisico occupato da tali valori mancanti (**0 byte su disco**) grazie all'omissione dinamica dei campi nulli nei documenti BSON. L'efficienza di archiviazione di MongoDB non deriva dunque da un'imputazione aggressiva e artificiale dei buchi residui, bensì dalla capacità di rappresentare serie temporali a frequenza mista senza pagare il costo di maschere di bit o padding;
2. **Preservazione della griglia temporale e impatto contenuto dello scarto righe:** grazie alla scelta di posticipare l'eliminazione delle righe interamente vuote (`dropna(how='all')`) al termine della catena di pulizia Silver, il processo di bonifica su 500 casi rimuove unicamente **162.998 righe su 5.565.384 campioni complessivi** presenti nel **Bronze**, pari ad appena il **2.93%** dei timestamp iniziali (5.402.386 record attivi mantenuti). Questa frazione marginale corrisponde esclusivamente ai minuti iniziali e finali di preparazione della sala operatoria in cui nessun sensore è ancora collegato al paziente, preservando intatta la continuità a 1 Hz durante l'intero atto chirurgico;
3. **Normalizzazione dell'asse temporale:** l'asse temporale è stato normalizzato sommando i secondi relativi dell'intervento a una data convenzionale fissa (`2020-01-01 + Time`). Sebbene tale espediente risponda pienamente alle esigenze di anonimizzazione, ordinamento e indicizzazione delle collezioni temporali, in una futura implementazione ospedaliera in tempo reale i campioni verranno naturalmente ancorati al clock sincronizzato NTP del monitor di sala.

CAPITOLO 4

Modelli di anomaly detection

4.1 Approccio ibrido e multilivello al rilevamento di anomalie

Nel monitoraggio intraoperatorio continuo, l'identificazione precoce di deviazioni fisiologiche critiche costituisce un obiettivo primario per la sicurezza del paziente. La dinamica biologica di un organismo sottoposto ad anestesia generale e a stimoli chirurgici è tuttavia estremamente complessa ed eterogenea: gli eventi avversi possono manifestarsi sia sotto forma di repentine cadute pressorie sia come graduali e insidiose dissociazioni tra parametri vitali multipli.

Per intercettare questo ampio spettro di condizioni, il presente progetto non si affida a un'unica tecnica di analisi, bensì implementa un approccio ibrido multilivello fondato su quattro metodologie complementari: due regole cliniche deterministiche dominio-specifiche, formulate sulla base delle evidenze della letteratura medica (lo **Shock Index** e la condizione di ipotensione severa); due algoritmi di **machine learning** non supervisionato capaci di operare sullo spazio congiunto delle serie storiche, segnatamente l'algoritmo ad alberi **Isolation Forest** e un modello neurale ricorrente basato su **LSTM Autoencoder**.

La complementarità di questo schema risponde a una precisa motivazione clinica e ingegneristica: le regole cliniche hanno una solida giustificazione fisiopatologica e forniscono riscontri trasparenti e immediatamente comprensibili dal personale medico, ma falliscono nell'identificare anomalie multivariate sfumate che precedono l'evento conclamato; al contrario, i modelli di **machine learning** non supervisionato eccellono nel riconoscere correlazioni inattese e configurazioni atipiche ad alta dimensionalità, sebbene necessitino di una contestualizzazione per distinguere gli artefatti statistici dalle vere emergenze cliniche.

4.2 Regole cliniche dominio-specifiche e filtri di freschezza

Le regole cliniche implementate nello script `src/detection/detector.py` operano sulle grandezze fisiologiche archiviate nel livello **Gold**, valutando soglie consolidate nella pratica anestesiologica. Prima di procedere al calcolo delle regole e delle feature, la funzione di caricamento `load_from_gold(db)` applica tre presidi fondamentali di **data governance**:

1. **Isolamento transazionale dei casi:** l'estrazione interroga preliminarmente la collezione `registry` filtrando esclusivamente i pazienti con stato `{"status": "committed"}`, escludendo qualsiasi serie incompleta;
2. **Mascheramento attivo degli outlier fisiologici:** poiché il livello **Silver** preserva i valori grezzi per finalità di audit limitandosi ad affiancare i flag booleani `quality_flags.*_outlier`, un artefatto meccanico (quale un picco di HR $= 320$ bpm dovuto al bisturi elettrico o una lettura pressoria nulla durante lo sgonfiaggio del bracciale) arriverebbe intatto al motore di calcolo, generando falsi allarmi deterministici. Per impedire tale distorsione, `load_from_gold()` ispeziona ciascun flag `*_outlier` e sostituisce in memoria i valori fuori scala con `np.nan` prima di invocare qualsiasi regola clinica o algoritmo di **machine learning**;
3. **Vincolo di freschezza clinica della pressione (`nibp_age_s` ≤ 120 s):** nel livello **Silver**,

le misurazioni intermittenti del bracciale pressorio vengono propagate in avanti tramite **forward-fill** fino a 300 s (5 minuti) per mantenere densa la matrice destinata ai modelli multivariati. Tuttavia, sul piano strettamente clinico, valutare una regola di emergenza acuta su un valore pressorio misurato quattro o cinque minuti prima risulterebbe metodologicamente scorretto: un singolo episodio ipotensivo transitorio produrrebbe infatti un allarme clinico continuo e artificiale per 300 secondi consecutivi. Per risolvere questa asimmetria, entrambe le regole cliniche interrogano il contatore `quality_flags.nibp_age_s` e impongono la soglia di freschezza massima `NIBP_MAX_CLINICAL_AGE_S = 120` secondi: una misura pressoria viene ritenuta clinicamente azionabile per le regole deterministiche solo se acquisita negli ultimi due minuti.

4.2.1 Shock Index clinico

Lo **Shock Index** (SI) è un indicatore emodinamico adimensionale ampiamente utilizzato nei reparti di rianimazione e terapia intensiva per l'identificazione precoce dello shock ipovolemico, emorragico o distributivo. Esso è definito matematicamente come il rapporto tra la frequenza cardiaca (HR) e la pressione arteriosa sistolica (SBP):

$$SI = \frac{\text{Solar8000_HR}}{\text{Solar8000_NIBP_SBP}}$$

In un soggetto adulto sano in condizioni stazionarie, il valore fisiologico dello **Shock Index** oscilla tra 0.5 e 0.7. Quando la gittata sistolica cala a seguito di un'emorragia intraoperatoria o di una vasodilatazione indotta dagli anestetici, il sistema nervoso autonomo tenta di compensare instaurando una tachicardia riflessa mentre la pressione sistolica flette. La funzione `compute_shock_index()` contrassegna come anomalo qualsiasi istante temporale in cui il rapporto superi la soglia critica in presenza di un dato pressorio recente:

$$SI > 0.9 \quad \wedge \quad \texttt{nibp_age_s} \leq 120 \text{ s}$$

Il superamento di questa soglia testimonia l'esaurimento della riserva fisiologica di compenso e segnala un'instabilità emodinamica che richiede un'immediata infusione volemica o la somministrazione di farmaci vasopressori.

4.2.2 Ipotensione severa concomitante a ipossiemia

La seconda euristica clinica, implementata nella funzione `compute_severe_hypotension()`, mira a identificare stati acuti di compromissione perfusiva associati a desaturazione periferica. La regola valuta la compresenza simultanea di una pressione arteriosa media insufficiente, di un'ipossiemia arteriosa e della validità temporale della misura NIBP:

$$\text{Solar8000_NIBP_MBP} < 65 \text{ mmHg} \quad \wedge \quad \text{Solar8000_PLETH_SPO2} < 90\% \quad \wedge \quad \texttt{nibp_age_s} \leq 120 \text{ s}$$

Il mantenimento di una pressione arteriosa media (MBP) al di sopra di 65 mmHg è universalmente riconosciuto nelle linee guida anestesiologiche come la soglia minima necessaria per preservare l'autoregolazione del flusso ematico cerebrale, coronarico e renale. Quando tale ipoperfusione si associa a una caduta della saturazione al di sotto del 90%, i tessuti entrano rapidamente in sofferenza ipossico-ischemica, rendendo la coincidenza temporale dei due segnali un allarme ad altissima priorità.

4.3 Ingegnerizzazione delle feature temporali

Per consentire agli algoritmi di **machine learning** di catturare non soltanto i valori istantanei dei parametri ma anche la dinamica evolutiva e la tendenza temporale delle serie biometriche, la procedura esegue una fase di estrazione di feature temporali derivate.

A partire dal segnale della frequenza cardiaca (`Solar8000/HR`), che presenta la cadenza di campionamento più continua e regolare a frequenza di 1 Hz, vengono calcolate tre feature dinamiche fondamentali. Indicando con x_t il valore della frequenza cardiaca all'istante temporale t (in secondi) e con $W = 5$ campioni l'ampiezza della finestra temporale retrospettiva, le metriche vengono definite formalmente come: la media mobile a cinque campioni (`HR_rolling_mean`), che attenua le micro-oscillazioni e gli artefatti ad alta frequenza senza introdurre ritardi significativi:

$$\mu_t = \frac{1}{W} \sum_{k=0}^{W-1} x_{t-k}$$

la deviazione standard mobile a cinque campioni (`HR_rolling_std`), che quantifica la dispersione e l'instabilità istantanea del battito cardiaco nel brevissimo termine:

$$\sigma_t = \sqrt{\frac{1}{W-1} \sum_{k=0}^{W-1} (x_{t-k} - \mu_t)^2}$$

la differenza prima (`HR_delta`), che misura la velocità istantanea di accelerazione o decelerazione del ritmo cardiaco:

$$\Delta x_t = x_t - x_{t-1}$$

L'introduzione di Δx_t riveste un ruolo fisiologico cruciale: una repentina variazione positiva di Δx_t (oltre 10–15 bpm in un singolo secondo) riflette una scarica adrenergica massiva indotta da un risveglio intraoperatorio o da uno stimolo algico chirurgico acuto, mentre un crollo improvviso testimonia un riflesso vagale o un blocco di conduzione atrioventricolare.

Al fine di garantire l'assoluta correttezza metodologica, il calcolo di queste finestre mobili viene vincolato al raggruppamento per singolo paziente mediante la trasformazione `df.groupby("case_id")`. Questo accorgimento impedisce rigorosamente che i confini temporali di un caso clinico si sovrappongano a quelli del caso successivo, scongiurando qualsiasi contaminazione analitica tra pazienti differenti nei punti di transizione.

Il vettore complessivo delle feature impiegato dai modelli di **machine learning** comprende otto grandezze numeriche: le cinque metriche vitali originarie (frequenza cardiaca, saturazione d'ossigeno e le tre componenti pressorie) affiancate dalle tre feature dinamiche derivate sulla frequenza. Al contrario, la variabile `nibp_age_s` calcolata nel livello **Silver** viene deliberatamente esclusa dal vettore; essa rappresenta infatti un metadato di governance attestante la freschezza temporale della misura e non una proprietà biologica intrinseca del paziente.

4.4 Modelli di machine learning non supervisionato

L'approccio basato sull'apprendimento non supervisionato prescinde dalla presenza di etichette a priori e mira a identificare pattern anomali nello spazio congiunto delle otto feature biometriche.

Prima di sottoporre la matrice delle feature agli algoritmi numerici (che non ammettono la presenza di valori `NaN`), entrambi i modelli eseguono una fase di imputazione di ripiego strettamente confinata all'interno del singolo intervento chirurgico:

```
df.groupby("case_id")[avail].ffill().groupby(df["case_id"]).bfill().fillna(0.0)
```

Questa doppia invocazione esplicita di `.groupby("case_id")` sia per il **forward-fill** sia per il **backward-fill** risolve una sottile ma insidiosa trappola implementativa di Pandas: poiché il metodo `.ffill()` applicato a un oggetto `GroupBy` restituisce un `DataFrame` ordinario non più raggruppato, concatenare direttamente `.bfill()` senza rieseguire il raggruppamento comporterebbe il riempimento

dei valori iniziali mancanti del paziente k con i primi valori validi del paziente $k+1$ (**data leakage inter-paziente**). Vincolando entrambe le direzioni al gruppo del singolo paziente — e ricorrendo a `0.0` esclusivamente qualora un canale sia interamente assente per tutta la durata della chirurgia — si garantisce un isolamento matematico totale tra i casi clinici, verificato formalmente dal test unitario `test_imputation_no_bfill_leakage_across_cases`.

4.4.1 Isolation Forest e semantica della soglia di contaminazione

Il primo algoritmo impiegato è l'**Isolation Forest**, un metodo di ensemble basato su alberi decisionali isolanti (**iTree**) particolarmente efficace per serie storiche ad alta dimensionalità.

Il principio teorico dell'algoritmo sfrutta le proprietà geometriche delle anomalie: i punti anomali sono pochi rispetto alla popolazione complessiva e tendono a posizionarsi in regioni periferiche e poco dense dello spazio vettoriale. L'algoritmo partiziona ricorsivamente lo spazio campionario selezionando casualmente una feature ed effettuando un taglio uniforme tra il valore minimo e massimo osservato. Poiché i campioni normali sono densamente raggruppati, essi richiedono un numero elevato di partizioni per essere isolati all'interno di una foglia; al contrario, le istanze anomale, trovandosi isolate nello spazio, vengono separate con pochissimi tagli decisionali.

La lunghezza media del percorso (**path length**) dalla radice alla foglia attraverso la foresta di alberi fornisce una misura inversa dello score continuo di anomalia. Nello script di produzione, l'algoritmo viene istanziato con una quota di contaminazione fissata al 5% (`contamination=0.05`) e con un seme deterministico fissato a 42 per garantire la riproducibilità dei risultati.

Dal punto di vista metodologico, è importante chiarire la semantica di questo iperparametro: fissare `contamination=0.05` (così come fissare la soglia al 95° percentile nell'Autoencoder) significa utilizzare il modello non supervisionato come un **motore di ordinamento continuo di severità (*anomaly ranking*)** su cui viene applicato un taglio operativo che seleziona il 5% di istanti temporali globalmente più estremi dell'intera coorte. Pertanto, la frazione globale del 5% sull'intero dataset è una scelta progettuale volta a bilanciare sensibilità e fatica da allarme, mentre la concentrazione specifica di tali anomalie all'interno del singolo paziente (che può variare dallo 0.2% in un intervento stabile a oltre il 25% in un paziente emodinamicamente critico) costituisce la reale informazione diagnostica estratta dal modello.

4.4.2 LSTM Autoencoder neurale (PyTorch) e scalabilità zero-copy

Per catturare le interdipendenze sequenziali e la memoria a lungo termine dei segnali fisiologici, è stato progettato e implementato un modello di deep learning basato su reti ricorrenti: l'**LSTM Autoencoder**, realizzato mediante la libreria PyTorch.

Come schematizzato nella Figura 4.1, l'architettura elabora le serie storiche mediante una struttura simmetrica di tipo **encoder-decoder**: il modello riceve in ingresso un tensore tridimensionale di forma (B, W_s, C), dove B rappresenta la dimensione del batch, $W_s = 30$ campioni (corrispondenti a 30 secondi di telemetria continua) è l'ampiezza della finestra scorrevole e $C = 8$ è il numero di canali biometrici standardizzati con punteggio z; l'encoder è costituito da uno strato ricorrente LSTM a 32 unità nascoste (`hidden_size=32`), che riceve la sequenza temporale $X \in \mathbb{R}^{30\times 8}$ e la comprime progressivamente nel vettore di stato nascosto finale $h_{\text{enc}} \in \mathbb{R}^{32}$. Tale vettore costituisce il **context vector** compresso, che riassume l'intera dinamica evolutiva del blocco temporale; per alimentare il decoder, il vettore h_{enc} viene replicato per 30 passi temporali consecutivi (**repeat vector**), fornendo a ciascun istante del processo di ricostruzione la medesima rappresentazione globale del contesto; il decoder è composto da un secondo strato ricorrente LSTM speculare a 32 unità, che elabora la sequenza riprodotta generando gli stati di decodifica $h_t^{\text{dec}} \in \mathbb{R}^{32}$ per ciascuno dei 30 istanti; un layer di proiezione lineare finale (`nn.Linear`), posizionato all'uscita del decoder, proietta ciascuno stato nascosto nello spazio continuo origi-

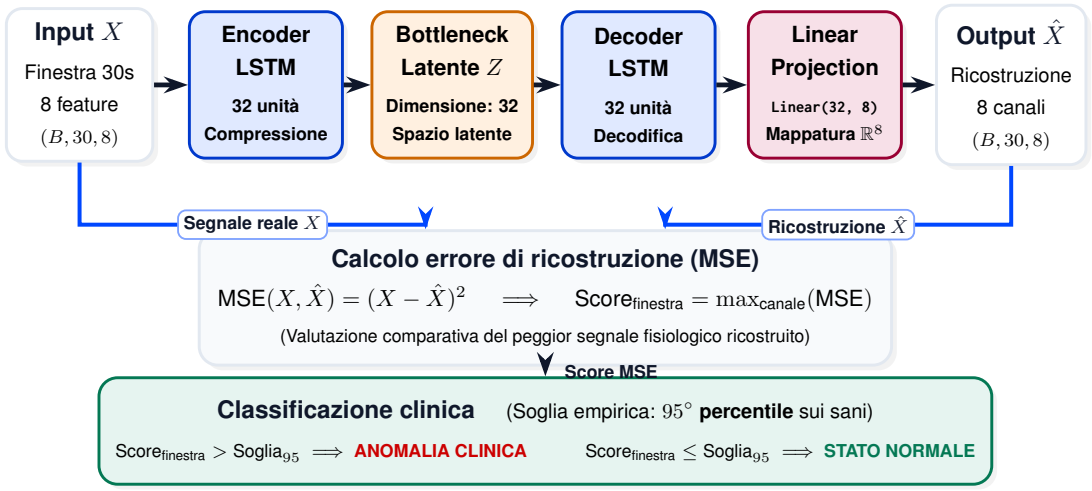


Figura 4.1: Architettura vettoriale della rete neurale LSTM Autoencoder con proiettore lineare in TikZ.

nale delle feature: $\hat{X}_t = W_p h_t^{\text{dec}} + b_p$, con pesi $W_p \in \mathbb{R}^{8\times32}$ e bias $b_p \in \mathbb{R}^8$. L'introduzione di questo strato lineare si è rivelata determinante: la funzione di attivazione tangente iperbolica nativa delle celle LSTM limita infatti le uscite all'intervallo ristretto $[-1, 1]$, impedendo la corretta ricostruzione di variabili standardizzate con punteggio z che superano tali estremi. Il proiettore lineare elimina questo vincolo consentendo la ricostruzione sull'intero asse reale.

L'architettura della rete in PyTorch, definita mediante la funzione interna `_make_lstm_autoencoder_class` nel modulo `detector.py`, è riportata nel Listato 4.1.

```
// Modulo PyTorch implementato in src/detection/detector.py
class LSTMAutoencoder(nn.Module):
    def __init__(self, n_features=8, hidden_size=32, n_layers=1):
        super().__init__()
        # Encoder: comprime la sequenza (B, 30, 8) nello stato nascosto (B, 32)
        self.encoder = nn.LSTM(n_features, hidden_size, n_layers, batch_first=
    True)
        # Decoder: ricostruisce la sequenza temporale partendo dal vettore
    latente
        self.decoder = nn.LSTM(hidden_size, hidden_size, n_layers, batch_first=
    True)
        # Proiettore lineare: mappa lo stato nascosto R^32 nello spazio
    originale R^8
        self.out = nn.Linear(hidden_size, n_features)

    def forward(self, x):
        # x ha forma (batch_size, sequence_length=30, n_features=8)
        _, (hidden, _) = self.encoder(x)
        # Replicazione del context vector per ciascuno dei 30 timestep
        context = hidden.permute(1, 0, 2).repeat(1, x.size(1), 1)
        output, _ = self.decoder(context)
        return self.out(output)  # dimensione ricostruita: (batch, 30, 8)
```

Listing 4.1: Definizione del modello PyTorch LSTM Autoencoder con proiettore lineare per l'anomaly detection.

Il passaggio dalla sperimentazione preliminare su 50 casi alla coorte completa di **500 casi clinici** (**5.402.386 punti temporali**) ha imposto una riprogettazione profonda della gestione della memoria nella classe `LSTMAutoencoderDetector`. Se infatti le viste generate da `sliding_window_view`

per ciascun paziente venissero concatenate globalmente tramite `np.concatenate` in un unico tensore tridimensionale $(5.387.886 \times 30 \times 8)$, NumPy sarebbe costretto a materializzare fisicamente in memoria RAM oltre **5.17 GB** di dati in precisione `float32` (o oltre **10.3 GB** in `float64`), causando l'esaurimento della memoria del container Docker (*Out-Of-Memory*) o un degrado drammatico delle prestazioni per paginazione su disco.

Per garantire un'esecuzione rapida e ad impronta di memoria costante ($\mathcal{O}(1)$ rispetto al numero di finestre), è stata ingegnerizzata la procedura `_make_window_views_per_case`:

- la matrice delle 8 feature viene normalizzata globalmente tramite `StandardScaler` e convertita immediatamente in precisione singola `np.float32`;
- per ciascun caso clinico, `np.lib.stride_tricks.sliding_window_view` costruisce una vista virtuale zero-copy $(N_i - 29, 30, 8)$ che manipola esclusivamente gli **strides** dei puntatori di memoria senza allocare nuovi buffer, salvando in una lista leggera le tuple descrittive `(case_id, win_view, end_indices)`;
- per la fase di addestramento, l'algoritmo campiona uniformemente un massimo di `MAX_TRAIN_WINDOWS = 15.000` finestre attraverso tutti i pazienti, materializzando in RAM unicamente i 15.000 blocchi estratti (appena **14.4 MB** di memoria) e addestrando la rete per 10 epoche con ottimizzatore Adam (lr $= 10^{-3}$, `BATCH_SIZE = 256`);
- durante la fase di inferenza sui 5.4 milioni di punti, la rete non concatena mai l'intero dataset, ma scorre sequenzialmente le viste dei singoli pazienti valutando blocchi contigui di `INFER_BATCH = 4.096` finestre alla volta (`torch.no_grad()`). Questa architettura ha permesso di abbattere il tempo di esecuzione completo sui 500 casi da decine di minuti di stallo in memoria a circa **80 secondi** su CPU.

Durante la fase di inferenza, la rete valuta la discrepanza tra la finestra di ingresso X e la corrispondente finestra ricostruita $\hat{X}$. Nel calcolo dell'errore di ricostruzione complessivo della finestra, si è deliberatamente scelto di non utilizzare la media aritmetica dell'errore quadratico medio su tutti gli otto canali. In ambito emodinamico, la mediazione globale introdurrebbe infatti un grave rischio di diluizione: se sette parametri su otto rimangono stabili, una caduta acuta e drammatica di un singolo canale vitale (quale una grave desaturazione SpO_2 al di sotto dell'80% per ostruzione respiratoria improvvisa) verrebbe attenuata dall'effetto mediato degli altri segnali, non riuscendo a superare la soglia di allarme. Per scongiurare tale evenienza, l'errore della finestra viene definito come il massimo errore quadratico medio osservato tra i canali biometrici:

$$\mathcal{E}(X) = \max_{c \in \{1,\dots,C\}} \left(\frac{1}{W_s} \sum_{t=1}^{W_s} (X_{t,c} - \hat{X}_{t,c})^2 \right)$$

L'errore così ottenuto viene assegnato con precisione all'ultimo istante temporale della finestra scorrevole. La soglia decisionale di anomalia viene fissata in modo non parametrico al 95° percentile della distribuzione empirica degli errori $\mathcal{E}(X)$ su tutte le finestre valutate.

Principio di sensibilità emodinamica e finestre Zero-Copy

L'impiego del massimo errore quadratico medio canale per canale ($\max_c \text{MSE}_c$) garantisce che il modello neurale mantenga una sensibilità elevata anche di fronte a collassi isolati di un singolo parametro vitale. Contestualmente, l'elaborazione a viste **zero-copy** per singolo paziente evita l'allocazione di oltre 5 GB di tensori 3D, permettendo di analizzare 5.4 milioni di secondi intraoperatori in poco più di un minuto.

4.5 Valutazione empirica, metriche e concordanza

La valutazione delle prestazioni dei modelli di **machine learning** in ambito clinico su serie storiche ad alta frequenza incontra un ostacolo metodologico significativo: l'assenza, nel dataset pubblico VitalDB, di annotazioni manuali second-by-second certificate da un anestesista umano.

Per condurre una valutazione quantitativa rigorosa e riproducibile, nello script `src/analysis/evaluate_models.py` è stato adottato un protocollo di concordanza clinica basato su **pseudo-ground truth**: l'unione logica degli eventi critici intercettati dalle due regole cliniche dominio-specifiche (**Shock Index** > 0.9 oppure ipotensione severa MBP $< 65 \wedge \text{SpO}_2 < 90\%$, entrambe vincolate alla freschezza pressoria `nibp_age_s` ≤ 120 s) è stata assunta come etichetta di riferimento per calcolare le metriche di classificazione. Inoltre, sul piano dell'implementazione dello script di valutazione, è stata eliminata un'inefficienza presente nelle prime versioni in cui l'**Isolation Forest** veniva addestrato due volte separatamente (una per le predizioni binarie e una per gli score continui): ora lo script esegue un **singolo fit** del modello, ricavando dalla medesima istanza sia le classi predette (`fit_predict(X)`) sia il punteggio continuo di anomalia (`-model.score_samples(X)`) destinato al calcolo della curva Precision-Recall (`average_precision_score`).

Su un totale di **5.402.386 punti temporali** appartenenti ai **500 casi clinici** con stato `committed`, il protocollo ha censito **262.978 istanti di positività clinica**, corrispondenti a una prevalenza di base (**base rate**) pari al **4.87%**.

Questa marcata asimmetria tra le classi (95.13% di secondi stabili contro 4.87% di eventi critici) impone una riflessione metodologica sul noto **Accuracy Paradox**: un classificatore banale che predica sistematicamente la classe maggioritaria ("normale" per ogni secondo dell'intervento) ottiene un'accuratezza globale apparentemente eccellente del **95.13%**, pur risultando totalmente inutile in sala operatoria avendo mancato il 100% delle emergenze. Per tale ragione, l'accuratezza globale è stata esclusa dai criteri di giudizio a favore di metriche specifiche per classi sbilanciate: **Precision**, **Recall**, **F1-score** sulla classe anomala e **PR-AUC** (*Average Precision*), il cui valore di riferimento casuale coincide esattamente con la prevalenza positiva (0.0487). I risultati sperimentali sui 500 casi, salvati in `data/ml_evaluation.json` con tracciabilità del `run_id` e della versione della pipeline, sono compendiati nella Tabella 4.1.

Modello / Algoritmo	Precision (anom.)	Recall (anom.)	F1-score (anom.)	PR-AUC (Average Precision)	Guadagno su Baseline
Classificatore banale (Baseline)	0.000	0.000	0.000	0.049 (Base rate 4.87%)	1.0× (Caso)
Isolation Forest	**0.124**	**0.127**	**0.126**	**0.105**	**2.2× sopra il caso**
LSTM Autoencoder	0.096	0.098	0.097	0.067	**1.4× sopra il caso**

Tabella 4.1: Valutazione comparativa dei modelli ML rispetto alla pseudo-ground truth clinica su 500 casi (5.402.386 record).

I risultati numerici riportati nella Tabella 4.1 — e accompagnati nel file `ml_evaluation.json` da un campo esplicito `notes` di trasparenza metodologica — evidenziano che entrambi i modelli ordinano gli eventi critici significativamente meglio di un selettore casuale: l'**Isolation Forest** raggiunge una PR-AUC di **0.105** (circa **2.2 volte** la baseline casuale di 0.049, con F1-score pari a 0.126), mentre l'**LSTM Autoencoder** ottiene una PR-AUC di **0.067** (circa **1.4 volte** la baseline, con F1-score pari a 0.097).

Una disamina scientifica rigorosa permette di spiegare con esattezza sia il limite intrinseco di questo benchmark sia il motivo per cui i valori assoluti di F1-score (~ 0.10–0.13) sono fisiologicamente attesi e desiderabili in un sistema ibrido:

1. **Natura euristica della Pseudo-Ground Truth:** le etichette di riferimento non provengono da diagnosi cliniche indipendenti esterne ai dati, ma sono costruite algebricamente sulle stesse tracce (`HR`, `SBP`, `MBP`, `SPO2`) fornite in ingresso ai modelli. La metrica misura pertanto

il **grado di concordanza (*overlap*)** tra l'euristica medica bivariata e la geometria non supervisionata dei modelli, e non la capacità predittiva assoluta verso un gold standard umano;

2. **Divergenza strutturale tra regole statiche e spazio multivariato dinamico:** le regole cliniche valutano condizioni istantanee su coppie di variabili grezze e vengono silenziate quando la misura pressoria ha più di 120 secondi (`nibp_age_s > 120`). Al contrario, **Isolation Forest** e **LSTM Autoencoder** operano in uno spazio a 8 dimensioni che include la variabilità del ritmo cardiaco (`HR_rolling_std`), la derivata prima (`HR_delta`) e — nel caso dell'Autoencoder — la dinamica ricorrente su finestre di 30 secondi, continuando a presidiare il paziente anche negli intervalli in cui il bracciale pressorio è inattivo;
3. **Valore clinico delle segnalazioni non sovrapposte:** i punti segnalati dai modelli ML che non coincidono con la pseudo-ground truth non rappresentano meramente dei falsi positivi, bensì intercettano pattern prodromici e multi-parametrici che le soglie statiche ignorano per costruzione:
    - micro-instabilità aritmiche caratterizzate da picchi improvvisi di `HR_rolling_std` a pressione arteriosa ancora normale;
    - brusche decelerazioni vagali ($\Delta x_t < -15$ bpm/s) indotte da trazioni chirurgiche viscerali;
    - cadute pressorie rapide ma sub-soglia (es. MBP che precipita da 95 a 67 mmHg in due cicli), che l'Autoencoder riconosce come rottura della traiettoria temporale prima ancora che venga varcato il muro statico dei 65 mmHg.

Se i modelli di **machine learning** avessero restituito una PR-AUC prossima a 1.0, essi sarebbero risultati ridondanti, limitandosi a replicare con un costo computazionale maggiore due semplici disuguaglianze algebriche. La concordanza parziale ma statisticamente superiore al caso ($2.2\times$ e $1.4\times$) dimostra invece la reale complementarità dell'architettura multilivello.

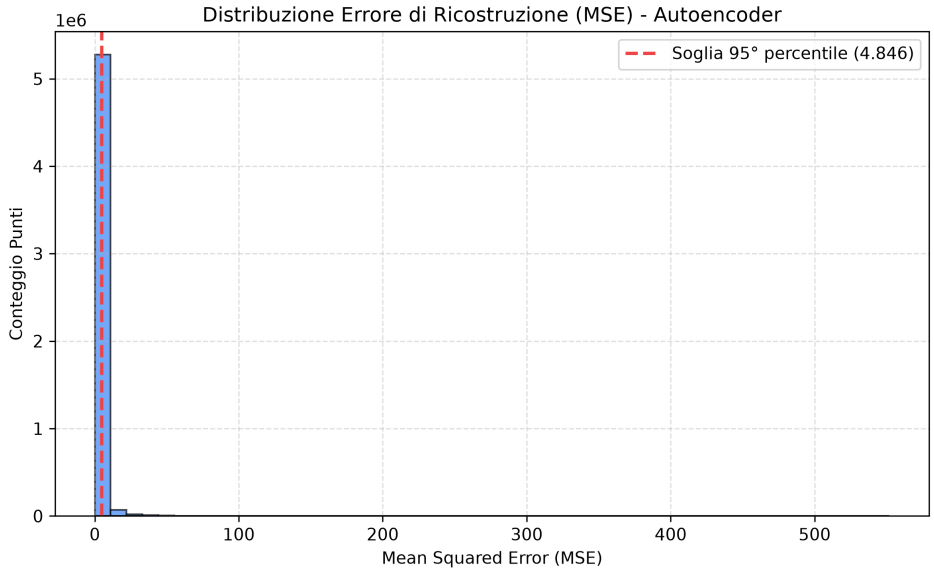


Figura 4.2: Distribuzione dell'errore di ricostruzione (MSE) dell'autoencoder con soglia al $95°$ percentile.

L'efficacia della modellazione neurale trova conferma nell'analisi della distribuzione dell'errore di ricostruzione dell'**Autoencoder**, illustrata nella Figura 4.2. La quasi totalità dei punti temporali fisiologicamente stabili si concentra in una campana ristretta con valori di errore quadratico

medio prossimi allo zero; al contrario, le configurazioni atipiche generano una marcata coda lunga a destra della distribuzione, superando nettamente la soglia empirica del 95° percentile evidenziata dalla linea tratteggiata.

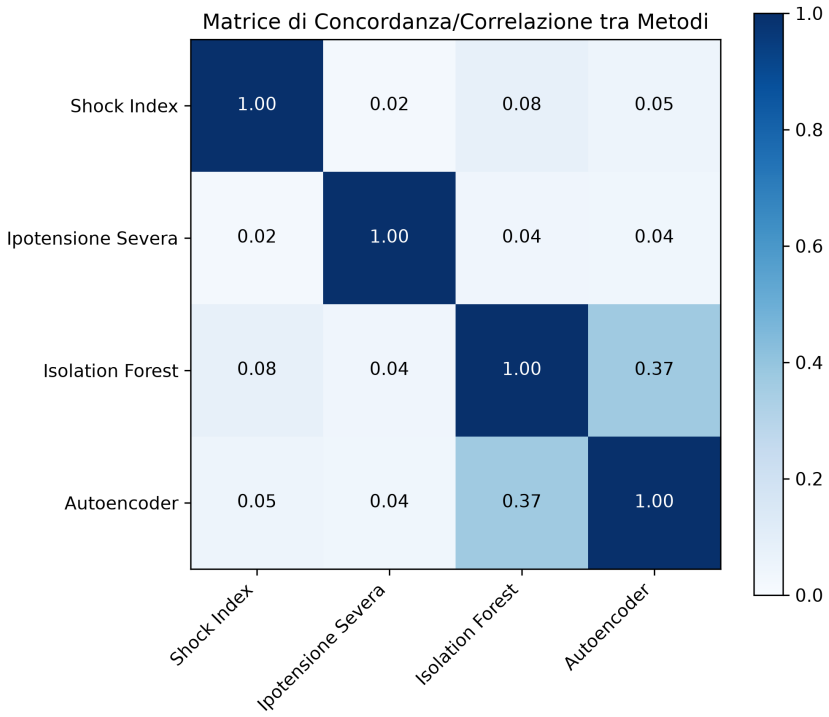


Figura 4.3: Matrice di concordanza/correlazione tra le regole cliniche e i modelli ML.

La reciproca interazione tra le quattro metodologie di detection è visualizzata nella matrice di concordanza e correlazione di Figura 4.3. Il grafico evidenzia la sinergia tra le regole emodinamiche e i modelli non supervisionati, confermando come l'approccio ibrido consenta di coniugare la specificità dell'euristica medica con la sensibilità predittiva del **machine learning**.

4.6 Persistenza sicura delle anomalie su MongoDB (Insert-Before-Delete)

A completamento del flusso di rilevamento, gli istanti temporali valutati come anomali da almeno una delle quattro metodologie vengono persistiti all'interno della collezione MongoDB denominata `anomalies_detected`.

Ciascun documento memorizzato nella collezione raccoglie le seguenti informazioni: l'identificativo del caso clinico (`case_id`); il timestamp esatto in cui si è verificata l'anomalia; l'array `methods`, che elenca i nomi degli algoritmi concordanti su quell'istante (ad esempio `["shock_index", "isolation_forest"]`); la marcatura temporale di esecuzione `detected_at`, che certifica l'istante di generazione del risultato analitico.

Nelle prime iterazioni del progetto, l'idempotenza del salvataggio veniva perseguita cancellando preliminarmente tutte le anomalie pregresse dei casi analizzati (`delete_many`) prima di inserire i nuovi risultati. Tuttavia, su un'esecuzione massiva da 500 casi, tale approccio esponeva al rischio di perdita di dati: se il processo si fosse interrotto durante la scrittura dei nuovi blocchi, la collezione `anomalies_detected` sarebbe rimasta priva sia dei vecchi sia dei nuovi riscontri.

Per garantire una sostituzione atomica e senza disservizi (*zero-downtime replacement*), in `src/detection/detector`

è stato implementato il pattern **Insert-Before-Delete con versionamento temporale**:

1. all'avvio della fase di persistenza viene generato un unico timestamp di batch `now = datetime.now(timezone.utc)`, assegnato al campo `detected_at` di tutti i nuovi record anomali;
2. i nuovi documenti vengono inseriti nella collezione `anomalies_detected` a blocchi controllati di 20.000 unità (`insert_many`);
3. **soltanto dopo il completamento con successo di tutti gli inserimenti**, la procedura esegue una cancellazione selettiva che rimuove esclusivamente i record appartenenti ad esecuzioni precedenti per i casi analizzati:

   ```
   db["anomalies_detected"].delete_many({"case_id": {"$in": analyzed_cases}, "detected_at": {"$ne": now}}
   ```

   (ovvero una pulizia completa dei casi analizzati nel solo caso limite in cui la nuova esecuzione abbia accertato zero anomalie).

Grazie a questo meccanismo, un eventuale arresto imprevisto durante il calcolo o il salvataggio non lascia mai i pazienti privi di copertura diagnostica nella dashboard, preservando i risultati dell'esecuzione precedente fino al subentro completo della nuova generazione di anomalie.

CAPITOLO 5

Ingegnerizzazione del sistema, interfaccia grafica e conclusioni

5.1 Backend REST API con FastAPI

L'integrazione e la fruizione dei dati archiviati nel livello **Gold** di MongoDB e l'esecuzione degli algoritmi di rilevamento delle anomalie sono orchestrate da un backend sviluppato in linguaggio Python mediante il framework **FastAPI**.

La scelta di FastAPI è motivata dalle elevate prestazioni del suo motore asincrono basato su **Starlette** e **Pydantic**, che consentono la validazione rigorosa dei parametri di richiesta, la serializzazione nativa dei documenti BSON in formato JSON e la generazione automatica della documentazione interattiva OpenAPI (disponibile all'indirizzo `/docs`). Il servizio REST opera all'interno del container applicativo `vitaldb_api` sulla porta di rete 8000 ed è implementato nel file `api/main.py`.

Dal punto di vista dell'efficienza delle risorse, il backend gestisce la connettività verso MongoDB mediante un'unica istanza globale del client `pymongo.MongoClient`. Questo accorgimento implementa un pool di connessioni riutilizzabili (**connection pooling**) che mantiene aperti i socket TCP tra le varie richieste HTTP, azzerando il costo di negoziazione dell'handshake di rete per ciascuna chiamata all'API. Inoltre, prima della trasmissione dei dati sulla rete, il backend esegue una bonifica dei valori numerici: i valori non validi o mancanti (`np.nan`) vengono convertiti nel valore Python `None`, serializzato nel corrispondente valore `null` standard dello standard JSON. Questa sanitizzazione previene la generazione di token non standard `NaN`, che provocherebbero errori di decodifica bloccanti nei parser JavaScript dei browser moderni (`JSON.parse`).

L'interfaccia espone un insieme organico di rotte operative: l'endpoint `GET /health`, che svolge la funzione di monitoraggio dello stato di salute del container ed effettua un'interrogazione di ping al database. Esso restituisce un oggetto strutturato contenente lo stato della connettività e le statistiche aggregate di sistema: il numero di casi censiti nel `registry`, il totale dei punti temporali archiviati in `vital_signals` e il conteggio cumulativo delle anomalie salvate in `anomalies_detected`; l'endpoint `GET /cases`, che estrae l'elenco dei casi clinici certificati nel catalogo tramite la funzione `list_loaded_cases(db)`, filtrando rigorosamente i soli record con stato `{"status": "committed"}`. Per prevenire il classico degrado prestazionale dovuto al problema delle $N + 1$ query (in cui per ogni caso verrebbe effettuata una lettura distinta per recuperare reparto, età e sesso del paziente), il metodo adotta un'unica pipeline di aggregazione MongoDB con operatori `$group` e `$first`. Questa ottimizzazione riduce l'intero recupero anagrafico dei 500 pazienti a due sole operazioni complessive sul database; l'endpoint `GET /cases/{case_id}/series`, che restituisce la serie temporale biometrica del paziente richiesto. L'endpoint supporta il parametro opzionale `window_seconds` (con valori selezionabili quali 30, 60 o 300 secondi): quando specificato, l'API attiva la pipeline di aggregazione temporale descritta nel capitolo precedente con operatore `$dateTrunc`, riducendo drasticamente il payload di rete trasferito al browser per tracciati che si estendono per molte ore; l'endpoint `POST /cases/{case_id}/detect`, la cui implementazione reale è riportata nel Listato 5.1, innesca l'esecuzione on-demand dell'intera pipeline di **anomaly detection** sul paziente selezionato invocando `run_detection_pipeline(db, [case_id])`. La funzione applica automatica-

mente il mascheramento degli outlier, il filtro di freschezza pressoria (`nibp_age_s` ≤ 120 s) per le regole cliniche (**Shock Index** e ipotensione severa), l'inferenza dei modelli **Isolation Forest** e **LSTM Autoencoder**, e la persistenza sicura tramite pattern *Insert-Before-Delete* nella collezione `anomalies_detected`, restituendo al client sia il riepilogo per metodologia sia il dettaglio sanitizzato degli istanti anomali; gli endpoint dedicati alla trasparenza e alla governance, segnatamente `GET /quality`, `GET /benchmark` e `GET /evaluation`, che espongono direttamente in formato JSON i report di bonifica del livello **Silver**, i risultati del benchmark di storage e latenza, e le metriche quantitative di validazione dei modelli di **machine learning** (tutti corredati da `run_id` e `pipeline_version`).

```
// Endpoint implementato nel backend FastAPI (api/main.py)
@app.post("/cases/{case_id}/detect")
def detect_case(case_id: int):
    """Esegue l'algoritmo multilivello di Anomaly Detection sul caso specificato."""
    db = get_db()
    # Esegue regole cliniche (con nibp_age_s <= 120s), ML e salvataggio Insert-Before-Delete
    df = run_detection_pipeline(db, [case_id])

    if df.empty:
        raise HTTPException(
            status_code=404,
            detail=f"Nessun dato temporale disponibile per case_id={case_id}"
        )

    flag_cols = [c for c in df.columns if c.endswith("_anomaly")]
    anomalous_rows = df[df[flag_cols].any(axis=1)]

    summary_by_method = {
        col.replace("_anomaly", ""): int(df[col].sum())
        for col in flag_cols
    }

    anomalies = []
    for _, row in anomalous_rows.iterrows():
        methods = [c.replace("_anomaly", "") for c in flag_cols if bool(row[c])]
        hr_val = float(row["Solar8000_HR"]) if pd.notna(row.get("Solar8000_HR")) else None
        spo2_val = float(row["Solar8000_PLETH_SPO2"]) if pd.notna(row.get("Solar8000_PLETH_SPO2")) else None
        sbp_val = float(row["Solar8000_NIBP_SBP"]) if pd.notna(row.get("Solar8000_NIBP_SBP")) else None
        dbp_val = float(row["Solar8000_NIBP_DBP"]) if pd.notna(row.get("Solar8000_NIBP_DBP")) else None
        mbp_val = float(row["Solar8000_NIBP_MBP"]) if pd.notna(row.get("Solar8000_NIBP_MBP")) else None
        si_val = float(row["shock_index"]) if pd.notna(row.get("shock_index")) else None

        anomalies.append({
            "timestamp": str(row["timestamp"]),
            "methods": methods,
            "hr": hr_val, "spo2": spo2_val,
            "sbp": sbp_val, "dbp": dbp_val, "mbp": mbp_val,
```

```
            "shock_index": round(si_val, 3) if si_val is not None else None
        })

    return {
        "case_id": case_id,
        "anomaly_count": len(anomalies),
        "summary_by_method": summary_by_method,
        "anomalies": anomalies,
    }
```

Listing 5.1: Definizione della rotta REST per l'esecuzione della pipeline di anomaly detection in FastAPI.

Accanto agli endpoint analitici, FastAPI ospita due montaggi di file statici: la rotta `/dashboard`, che distribuisce l'intera interfaccia web interattiva, e la rotta `/presentazione`, che rende fruibili le slide illustrative del progetto per la discussione accademica.

5.2 Dashboard web e interfaccia utente interattiva

A corredo dell'infrastruttura dati è stata sviluppata un'interfaccia grafica web concepita per consentire al personale sanitario o all'analista di esplorare i tracciati biometrici, analizzare le anomalie e ispezionare i parametri di governance.

Come mostrato nella Figura 5.1, l'interfaccia è stata realizzata seguendo i principi di un'estetica moderna e funzionale. Dal punto di vista architetturale, la dashboard rifugge la complessità e la pesantezza di framework monolitici, impiegando standard aperti del web: un'architettura in html5 semantico, fogli di stile css3 con variabili a token per la gestione dinamica del tema chiaro e scuro (**light/dark mode**), e codice in javascript puro. L'aspetto tipografico è valorizzato da caratteri tipografici dedicati, tra cui **Figtree** per le intestazioni, **Space Grotesk** per i testi di interfaccia e **JetBrains Mono** per i valori numerici e i timestamp.

Per garantire modularità e manutenibilità senza richiedere strumenti di compilazione esterni quali Webpack o Vite, il codice dell'interfaccia adotta un sistema di caricamento modulare asincrono implementato nello script `dashboard/js/loader.js`. Il file principale `index.html` definisce dei contenitori dichiarativi denominati slot mediante l'attributo personalizzato `data-include` (ad esempio `<div data-include="partials/sidebar.html"></div>`). All'avvio dell'applicazione, la funzione `loadPartials` individua tutti gli slot mediante `querySelectorAll`, invia richieste HTTP parallele tramite `Promise.all` e l'API `fetch`, e provvede a sostituire ciascun tag con il contenuto HTML recuperato. Al fine di impedire che il browser mantenga in cache versioni obsolete dei file durante gli aggiornamenti, il caricatore accoda a ciascuna richiesta un parametro di versione esplicito (`?v=3.8`), realizzando una tecnica efficace di invalidazione della cache (**cache busting**).

La piattaforma si articola su due ambienti di lavoro principali, accessibili tramite selettore in testata: la vista di telemetria clinica e la vista di governance del **lakehouse**.

5.2.1 Vista di telemetria clinica e anomaly detection

Questo spazio di lavoro è dedicato al monitoraggio del paziente e alla rilevazione delle instabilità. L'ambiente si compone dei seguenti moduli integrati: la barra laterale (**sidebar**), che elenca tutti i 500 pazienti certificati nel livello **Gold** fornendo un campo di ricerca rapida e badge identificativi con reparto chirurgico, età e sesso; il **Patient HUD** (Heads-Up Display), un banner informativo collocato in cima alla pagina che visualizza l'identificativo del paziente selezionato, il reparto e i parametri anagrafici. Il modulo ospita i comandi operativi principali: il selettore della risoluzione temporale (che consente di visualizzare il dato grezzo a 1 secondo oppure aggregazioni a 30 secondi, 60 secondi o 5 minuti), il pulsante di ricarica e il pulsante per avviare il calcolo



Figura 5.1: Panoramica dell'interfaccia grafica web della dashboard per il monitoraggio e la governance.

delle anomalie; la **Vitals Grid**, una griglia a cinque tessere informative progettata secondo lo stile **bento-grid**. Ciascuna tessera ospita un indicatore visivo pulsante, il valore medio corrente e il range fisiologico di riferimento per la frequenza cardiaca ($60 - 100$ bpm), la saturazione d'ossigeno ($95 - 100\%$), la pressione arteriosa con valore medio (> 65 mmHg), lo **Shock Index** e il contatore cumulativo degli eventi anomali rilevati; la **Consensus Matrix**, definita nel file `partials/consensus-matrix.html`, che implementa un pannello interattivo a quattro schede corrispondenti ai quattro metodi di rilevamento (Shock Index, ipotensione severa, Isolation Forest e Autoencoder). Ciascuna scheda mostra un badge con il conteggio in tempo reale degli eventi individuati e consente di cliccare per isolare ed evidenziare sui grafici esclusivamente le anomalie prodotte da quel metodo specifico. Inoltre, ciascuna scheda integra un pulsante informativo che apre una finestra modale con la spiegazione fisiologica, la formula analitica e la soglia di allarme adottata; i tracciati telemetrici realizzati con la libreria **Chart.js**: due grafici interattivi sincronizzati ad asse temporale comune. Il grafico superiore traccia l'andamento congiunto di frequenza cardiaca e saturazione, mentre il grafico inferiore visualizza le tre componenti pressorie (sistolica, diastolica e media). I due grafici condividono la traccia di mira (**crosshair**) e i comandi di ingrandimento temporale, consentendo una correlazione visiva immediata tra alterazioni del ritmo e variazioni pressorie. Quando viene attivato il rilevamento, i punti anomali vengono evidenziati mediante marcatori cromatici sovrapposti alle curve dei segnali; la tabella analitica delle

anomalie, collocata a fondo pagina, che elenca cronologicamente ogni singolo secondo in cui è stata riscontrata un'irregolarità, riportando i valori biometrici istantanei, lo Shock Index calcolato e l'elenco degli algoritmi concordanti.

5.2.2 Vista di data lakehouse, governance e benchmark

La seconda schermata dell'interfaccia, orchestrata dal modulo `dashboard/js/lakehouse.js` e dal template `partials/lakehouse-view.html`, visualizza in modo trasparente le metriche di architettura dati e qualità del dato sincronizzate in tempo reale tramite le API REST:

- **Schede comparative di Storage e Query Latency:** illustrano tramite barre grafiche scalate dinamicamente sul massimo ingombro (`Math.max(bronzeMB, silverMB, goldMB)`) l'evoluzione dello spazio su disco sui 500 casi: dai **33.95 MB** del livello **Bronze**, passando per l'espansione del livello **Silver** a **44.52 MB** (`+31.1% con flag DQ`, motivata dall'aggiunta delle 12 colonne di governance e dalla densificazione LOCF), fino all'abbattimento nel livello **Gold** di MongoDB a **15.60 MB** (**-54.1%** rispetto a Bronze e **-65.0%** rispetto a Silver, con indicazione esplicita dell'overhead di **+2.56 MB** per gli indici B-Tree). Accanto allo storage viene rappresentato il fattore di accelerazione di **2.2 volte** nelle interrogazioni temporali (**12.02 ms** contro **26.95 ms**);
- **Tabella di valutazione dei modelli di Machine Learning:** sincronizzata dall'endpoint `GET /evaluation`, presenta il confronto tra **Isolation Forest** e **LSTM Autoencoder** esponendo Precision, Recall, F1-score e la colonna **PR-AUC (Average Precision)** (etichettata esplicitamente per riflettere l'uso di `average_precision_score` sugli score continui), calcolate sui **5.402.386 punti** della coorte Gold;
- **Tabella del Data Quality and Governance Report:** alimentata dall'endpoint `GET /quality`, elenca analiticamente per ciascun caso clinico i campioni grezzi del **Bronze** (5.565.384 totali), i campioni bonificati nel **Silver** (5.402.386 totali), le righe completamente vuote eliminate (162.998, pari al 2.93%), il tasso di valori mancanti e il conteggio degli outlier fisiologici rilevati per ciascun parametro vitale.

5.3 Containerizzazione Docker, test automatizzati e manutenzione

Al fine di garantire l'assoluta riproducibilità dell'ambiente software, l'indipendenza dal sistema operativo ospitante e la rapidità di dispiegamento in contesti ospedalieri, l'intero sistema è stato ingegnerizzato e containerizzato mediante Docker e Docker Compose.

L'intera architettura multi-livello del sistema, illustrata nella Figura 5.2, connette in modo organico l'interfaccia client, il gateway applicativo e il cluster di archiviazione MongoDB.

L'infrastruttura definita nel file `docker-compose.yml` orchestra due container interconnessi su una rete virtuale dedicata denominata `vitaldb_net`: il container del database `vitaldb_mongo`, basato sull'immagine ufficiale di MongoDB 8.0. Il servizio viene avviato configurando automaticamente un Replica Set a singolo nodo (`--replSet rs0`), prerequisito architetturale per abilitare le funzionalità avanzate di MongoDB. L'integrità dello storage è garantita da un volume persistente su disco, mentre le collezioni, gli schemi JSON e i metadati iniziali vengono predisposti automaticamente all'avvio mediante gli script montati nella directory di inizializzazione `/docker-entrypoint-initdb.d/`; il container applicativo `vitaldb_api`, costruito a partire da un'immagine leggera basata su Python 3.12-slim. Il container incorpora l'interprete dell'API FastAPI, il server ASGI `uvicorn` e l'insieme delle librerie matematiche necessarie all'esecuzione dei modelli, tra cui `pymongo`, `scikit-learn`, `pandas`, `numpy` e la versione per CPU di `torch`.

Un elemento architetturale di rilievo risiede nell'orchestrazione delle dipendenze basata su **heal-**

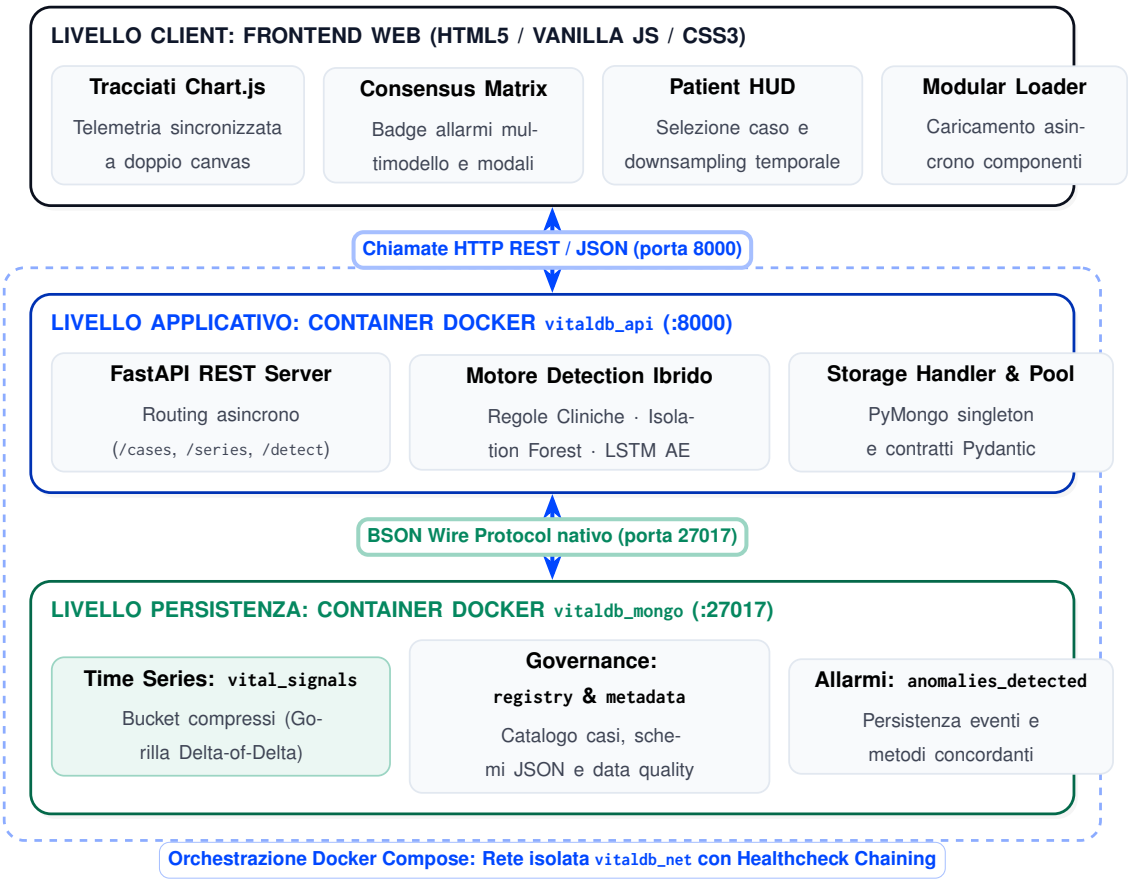


Figura 5.2: Architettura multi-livello completa dell'applicazione e containerizzazione Docker.

thcheck attivi. Poiché l'API richiede che il database non sia soltanto attivo a livello di processo ma abbia completato con successo l'inizializzazione del Replica Set e il caricamento degli schemi, il servizio `mongo` esegue un controllo ciclico mediante il client nativo `mongosh`. Lo script verifica che il nodo appartenga allo stato operativo primario o, in caso contrario, ne forza l'inizializzazione (`rs.initiate`). Nel file di composizione, il container `api` dichiara la dipendenza:

```
depends_on: { mongo: { condition: service_healthy } }
```

Questo meccanismo impedisce che il backend entri in esecuzione prima che il database sia pienamente operativo, azzerando qualsiasi fallimento di connessione all'avvio dello stack. L'intero sistema può quindi essere generato e avviato con un singolo comando:

```
docker compose up -build -d
```

Isolamento infrastrutturale e resilienza del deployment

L'orchestrazione Docker garantisce il disaccoppiamento totale tra il calcolo analitico e la persistenza dei dati: il volume permanente preserva le serie storiche e il catalogo di governance a fronte di riavvii del container, mentre la rete isolata `vitaldb_net` e il meccanismo di healthcheck chaining proteggono l'integrità del database durante l'avvio a freddo dello stack.

5.3.1 Verifica sperimentale automatizzata (Pytest) e strumenti di reset

Per garantire che le regole di **data governance**, le trasformazioni matematiche del layer **Silver** e gli algoritmi di **anomaly detection** mantengano una correttezza formale verificabile a ogni

modifica del codice, il progetto è stato corredato da una suite di **38 test unitari e d'integrazione** basata sul framework `pytest` (suddivisa nei moduli `tests/test_clean.py`, `tests/test_detection.py`, `tests/test_queries.py` e `tests/test_api.py`):

- **Test di bonifica e telemetria Silver (`test_clean.py`):** verificano che l'interpolazione lineare colmi esclusivamente i gap interni ≤ 5 s lasciando intatte le code finali prive di campioni successivi, che il rilevamento degli outlier avvenga prima del **forward-fill** senza propagare artefatti nelle righe successive, che i 5 flag `*_imputed` traccino separatamente ogni canale e che per ogni traccia sia sempre soddisfatta l'identità `observed` + `imputed` + `missing` $\equiv$ `rows_cleaned`;
- **Test clinici, isolamento ML e finestre ricorrenti (`test_detection.py`):** certificano il silenziamento dello Shock Index e dell'ipotensione severa quando la pressione ha un'età `nibp_age_s` > 120 s, l'assenza di finestre scorrevoli LSTM a cavallo tra pazienti diversi e l'assenza totale di **data leakage** nel `.bfill()` raggruppato per `case_id` (`test_imputation_no_bfill_leakage_acr`
- **Test di isolamento del catalogo Gold (`test_queries.py`):** verificano mediante mock della collezione `registry` che la funzione `list_loaded_cases()` interroghi rigorosamente il predicato `{"status": "committed"}`, escludendo i casi in stato `"loading"`, `"failed"` o privi di stato;
- **Test non distruttivi dell'API REST (`test_api.py`):** validano tutte le rotte HTTP di FastAPI. In particolare, per evitare che il test dell'endpoint `POST /cases/1/detect` sovrascriva o cancelli le anomalie reali prodotte dall'esecuzione dell'intera coorte nel database MongoDB di produzione, il test adotta una tecnica di **monkeypatching** (`monkeypatch.setattr(api_main, "run_detection_pipeline", fake_run_detection)`). Ciò consente di collaudare la logica di formattazione, sanitizzazione JSON e gestione degli errori dell'API in una frazione di secondo e in totale isolamento dal database.

Infine, a supporto dell'operatività ripetibile dell'intero flusso, è stato realizzato lo script di utilità `scripts/reset_pipeline.py`. Invocabile da riga di comando con il parametro `-layer` (`gold`, `silver`, `bronze` o `all`), lo strumento permette di azzerare in modo selettivo e controllato le collezioni MongoDB (`vital_signals`, `registry`, `anomalies_detected`) e i corrispondenti artefatti e report su disco, garantendo la possibilità di rieseguire da zero l'intera pipeline in condizioni perfettamente deterministiche.

5.4 Conclusioni, limiti e sviluppi futuri

Il lavoro svolto ha dimostrato su scala reale — attraverso l'elaborazione di **500 interventi chirurgici completi** per oltre **5.4 milioni di secondi di monitoraggio ad alta frequenza** — la fattibilità e l'efficacia di una piattaforma integrata che coniuga l'ingegneria dei dati temporali con il rilevamento ibrido delle anomalie intraoperatorie.

L'adozione della **Medallion Architecture** ha consentito di strutturare il flusso di raffinamento del dato clinico in stadi rigorosamente governati e collegati da un identificativo di esecuzione univoco (`run_id`): la persistenza immutabile e validata dei dati grezzi (**Bronze**, 33.95 MB), la bonifica conservativa con arricchimento a 18 colonne di metadati di qualità (**Silver**, 44.52 MB) e l'archiviazione compressa su MongoDB 8.0 Time Series (**Gold**, **15.60 MB** di storage puro più 2.56 MB di indici B-Tree). I benchmark quantitativi hanno certificato una riduzione dello spazio fisico dei dati del **65.0% rispetto al layer Silver** (e del **54.1% rispetto al Bronze**) e un'accelerazione di **2.2 volte** nei tempi di risposta delle query analitiche sul singolo paziente (**12.02 ms** contro **26.95 ms**).

Al contempo, l'approccio ibrido all'**anomaly detection** ha confermato il valore clinico di affianca-

re alle regole deterministiche dominio-specifiche (lo **Shock Index** e l'ipotensione severa, protette dal vincolo di freschezza pressoria `nibp_age_s` ≤ 120 s) due algoritmi di **machine learning** non supervisionato (**Isolation Forest** e **LSTM Autoencoder** con viste zero-copy), capaci di conseguire valori di PR-AUC pari a $2.2\times$ e $1.4\times$ la baseline casuale e di intercettare alterazioni multivariate prodromiche non visibili alle soglie statiche.

In un'ottica di rigore scientifico, è fondamentale riassumere i limiti intrinseci dell'architettura e le soluzioni di mitigazione adottate:

1. **Emulazione applicativa delle garanzie Lakehouse:** l'assenza di transazioni multi-documento e di validatori `$jsonSchema` nativi sulle **Time Series Collection** di MongoDB impedisce un'atomicità fisica ACID tra la collezione `vital_signals` e il catalogo `registry`. Questo limite strutturale è stato tuttavia neutralizzato a livello applicativo mediante il protocollo di caricamento a due fasi (`loading` $\rightarrow$ `committed` con recovery automatico all'avvio), la validazione preventiva dei contratti (`validate_records`) e il pattern *Insert-Before-Delete* per le anomalie;
2. **Natura della sparsità e della soglia non supervisionata:** la sparsità logica dei sensori dopo il **forward-fill** rimane pari al **26.3%** anche nel layer **Gold**, pur occupando **0 byte fisici** grazie all'omissione BSON dei campi nulli. Analogamente, nei modelli non supervisionati la quota del 5% (`contamination=0.05` e 95° percentile) opera come un selettore dei punteggi più estremi a livello di coorte; la validazione clinica definitiva richiede pertanto il confronto futuro con annotazioni mediche indipendenti;
3. **Ancoraggio temporale convenzionale:** l'asse temporale è normalizzato su una data di riferimento (`2020-01-01 + Time`) per tutelare l'anonimato dei pazienti del dataset pubblico VitalDB; in un dispiegamento ospedaliero reale i campioni utilizzeranno il timestamp assoluto di sala operatoria.

Da un punto di vista normativo e clinico, un sistema di supporto alle decisioni cliniche (**Clinical Decision Support System**, CDSS) destinato alla sala operatoria deve sottostare a stringenti requisiti di sicurezza. In ambito europeo, il Regolamento sui Dispositivi Medici (MDR 2017/745) classifica i software di monitoraggio diagnostico in classe IIa o IIb, imponendo la validazione prospettica della frequenza dei falsi allarmi per prevenire l'affaticamento da allarme (**alarm fatigue**) nel personale anestesiologico, unitamente alla piena conformità al GDPR mediante pseudonimizzazione e cifratura a riposo.

Tra le direttrici di sviluppo futuro più promettenti figurano:

- **Estensione granulare della Lineage MLOps a livello di singolo documento:** l'evoluzione del tracciamento di provenienza mediante l'inserimento diretto, all'interno di ciascun documento di `vital_signals` e `anomalies_detected`, di due identificativi distinti: il `pipeline_run_id` (che identifica la versione dei dati Gold) e il `detection_run_id` (che identifica la specifica esecuzione e configurazione di iperparametri dei modelli ML). Ciò consentirà di storicizzare e confrontare l'evoluzione predittiva di diverse versioni degli algoritmi sulla stessa coorte senza necessità di operazioni di join con il catalogo;
- **Streaming Data Engineering in tempo reale:** l'integrazione di broker di messaggistica distribuiti (quali Apache Kafka o Redpanda) per trasformare l'ingestione da micro-batch a flusso continuo sub-secondo direttamente interfacciato con i monitor di sala operatoria;
- **Ampliamento del pannello multiparametrico ed Explainable AI (XAI):** l'inclusione della pressione arteriosa cruenta invasiva (ART), della capnografia di fine espirazione ($EtCO_2$) e della profondità dell'anestesia (BIS), abbinata a tecniche di spiegabilità basate sui valori di Shapley (**SHAP**) per indicare istantaneamente al medico quali variabili abbiano

innescato l’allarme del modello neurale.